

IT4079:NGÔN NGỮ' và PHƯƠNG PHÁP DỊCH

Phạm Đăng Hải
haipd@soict.hust.edu.vn

Chương 5: Sinh mã

1. Sinh mã trung gian

2. Sinh mã đích

3. Tối ưu mã

Giới thiệu

- Bộ sinh mã trung gian chuyển chương trình nguồn sang chương trình tương đương trong ngôn ngữ trung gian
 - Chương trình trung gian là một chương trình cho một máy trừu tượng
- Ngôn ngữ trung gian được người thiết kế trình biên dịch quyết định, có thể là:
 - Cây cú pháp
 - Ký pháp Ba Lan sau (hậu tố)
 - Mã 3 địa chỉ ...

Nội dung

- Chương trình dịch định hướng cú pháp
- Cây cú pháp
- Ký pháp Ba lan sau
- Mã 3 địa chỉ
 - Các dạng mã
 - Dịch trực tiếp cú pháp thành mã 3 địa chỉ
 - Sinh mã cho khai báo
 - Sinh mã cho lệnh gán
 - Sinh mã cho các biểu thức logic
 - Sinh mã cho các cấu trúc lập trình

Chương trình dịch định hướng cú pháp

Mỗi ký hiệu VP liên kết với một tập thuộc tính:

– **Thuộc tính tổng hợp:**

- Giá trị của thuộc tính tại một nút trong cây được xác định từ giá trị của các nút con của nó.

– **Thuộc tính kế thừa:**

- Giá trị của thuộc tính được định nghĩa dựa vào giá trị nút cha và/hoặc các nút anh em của nó.

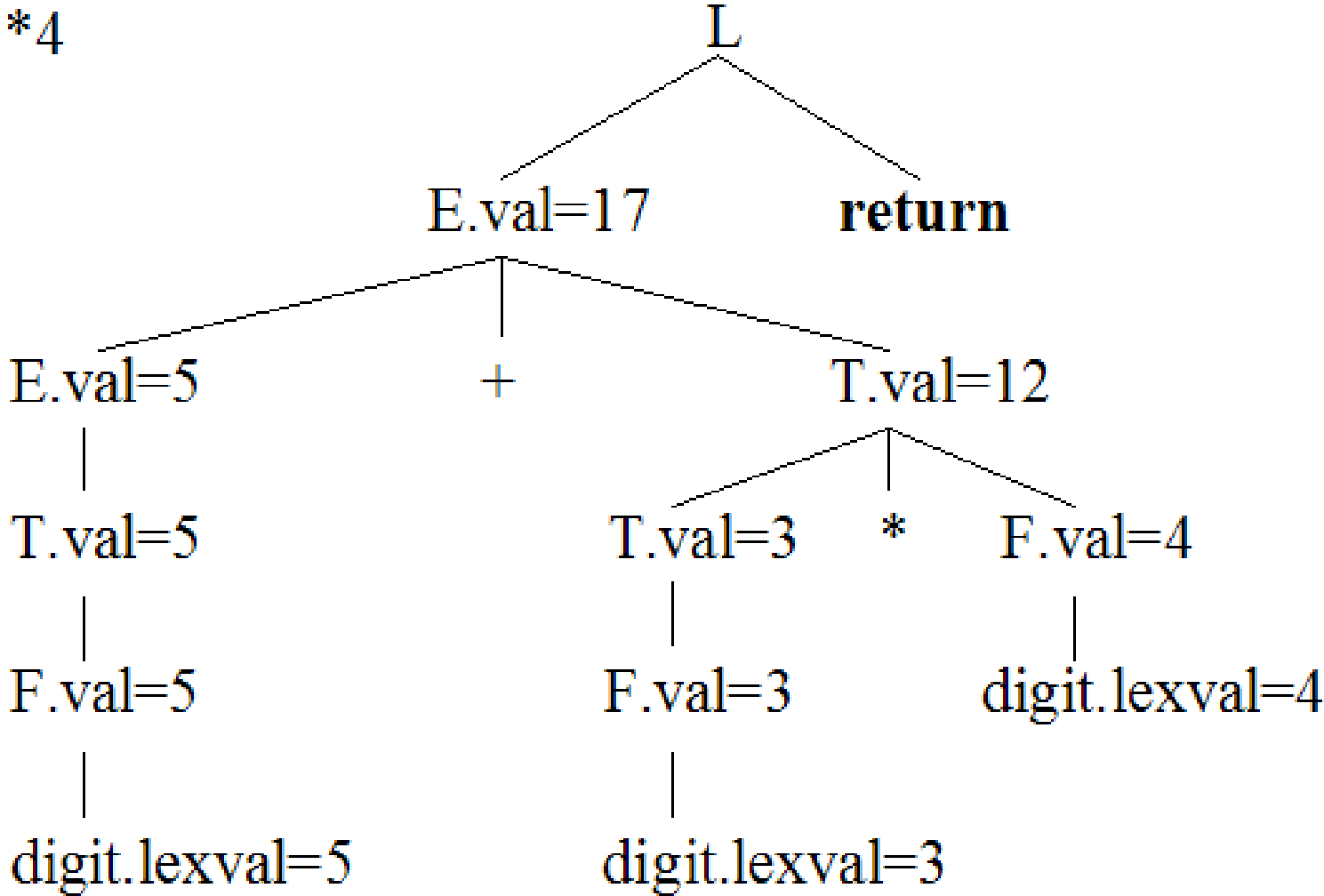
- Tồn tại một tập luật ngữ nghĩa dùng để tính giá trị thuộc tính

Ví dụ

Sản xuất	Quy tắc ngữ nghĩa
$L \rightarrow E \text{ return}$	Print ($E.val$)
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$
• Các ký hiệu <i>E, T, F</i> có thuộc tính tổng hợp <i>val</i> • Từ tố <i>digit</i> có thuộc tính tổng hợp <i>lexval</i> (Được bộ phân tích từ vựng đưa ra)	

Chú giải cây suy dẫn

Input: 5+3*4



Nội dung

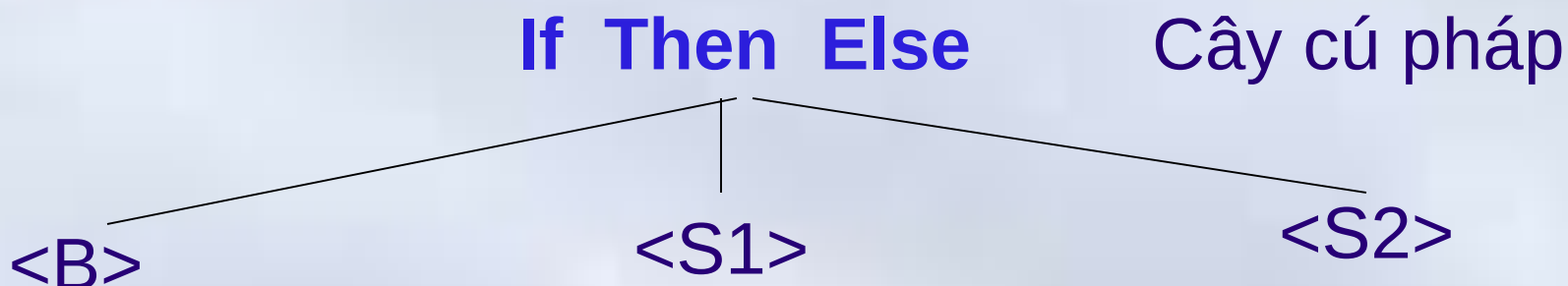
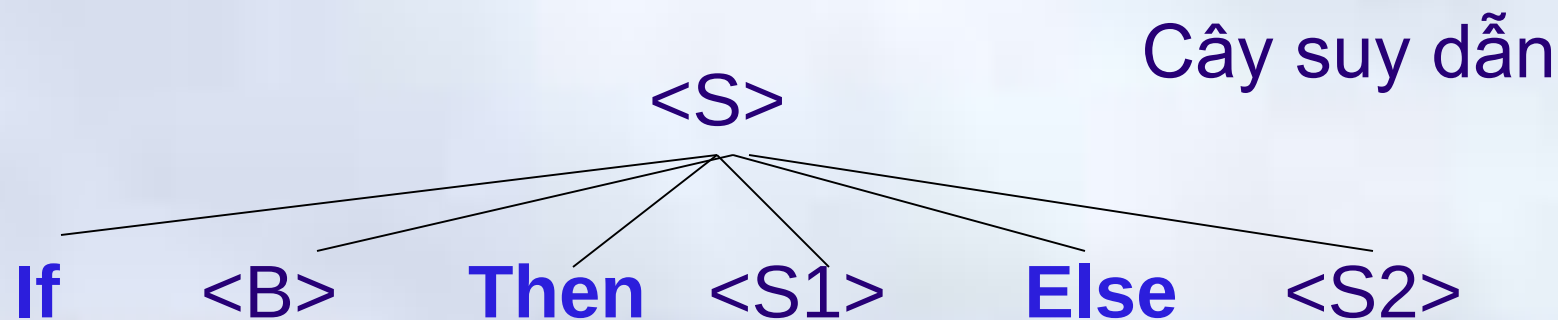
- Chương trình dịch định hướng cú pháp
- Cây cú pháp
- Ký pháp Ba lan sau
- Mã 3 địa chỉ
 - Các dạng mã
 - Dịch trực tiếp cú pháp thành mã 3 địa chỉ
 - Sinh mã cho khai báo
 - Sinh mã cho lệnh gán
 - Sinh mã cho các biểu thức logic
 - Sinh mã cho các cấu trúc lập trình

Cây cú pháp (Syntax tree)

- Cây cú pháp (*syntax tree*) là dạng thu gọn của cây phân tích (*parse tree*) dùng để biểu diễn cấu trúc của ngôn ngữ
- Trong cây cú pháp các toán tử và từ khóa không xuất hiện ở các nút lá mà đưa vào các nút trong.
 - Cha của các nút lá là các toán hạng tương ứng
- Cây cú pháp có ý nghĩa dụng trong cài đặt
 - Cây phân tích (cú pháp) chỉ ý nghĩa về mặt logic

Cây cú pháp $\rightarrow$ Ví dụ 1

Ví dụ: $S \rightarrow \text{If } B \text{ Then } S1 \text{ Else } S2$

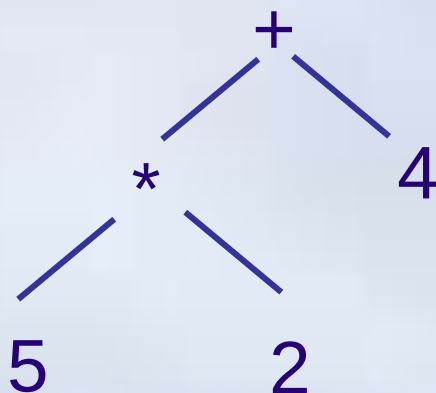


Xây dựng cây cú pháp → Ví dụ 2

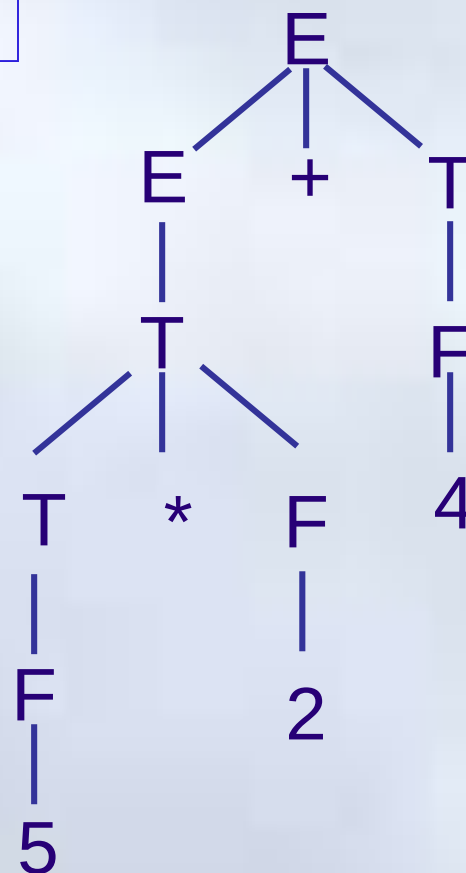
$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{num}$$

$$5 * 2 + 4$$


Cây cú pháp

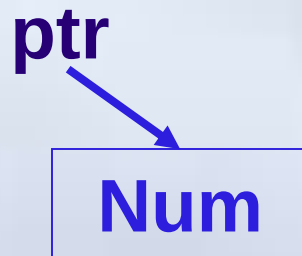


Cây phân tích

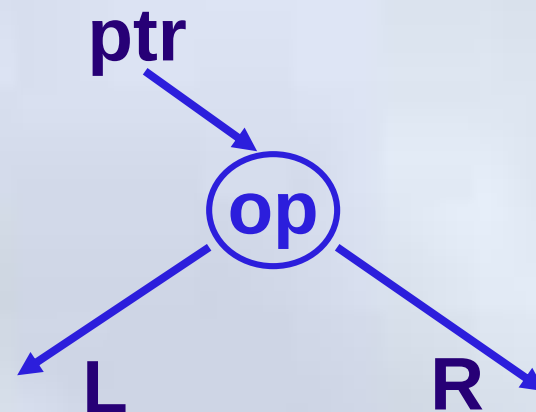
Xây dựng cây cú pháp cho biểu thức

- Các ký hiệu không kết thúc có thuộc tính tổng hợp **link** để lưu con trỏ, trỏ tới một nút trên cây cú pháp
- Sử dụng các hàm

`ptr = mkLeaf(Num)`



`ptr = mkNode(op, L, R)`



Cây cú pháp (Syntax tree)

Sản xuất	Luật ngữ nghĩa
$E \rightarrow E_1 + T$	$E.link := mkNode(+, E_1.link, T.link)$
$E \rightarrow T$	$E.link := T.link$
$T \rightarrow T_1 * F$	$T.link := mkNode(*, T_1.link, F.link)$
$T \rightarrow F$	$T.link := F.link$
$F \rightarrow (E)$	$F.link := E.link$
$F \rightarrow \text{num}$	$F.link := mkLeaf(num)$

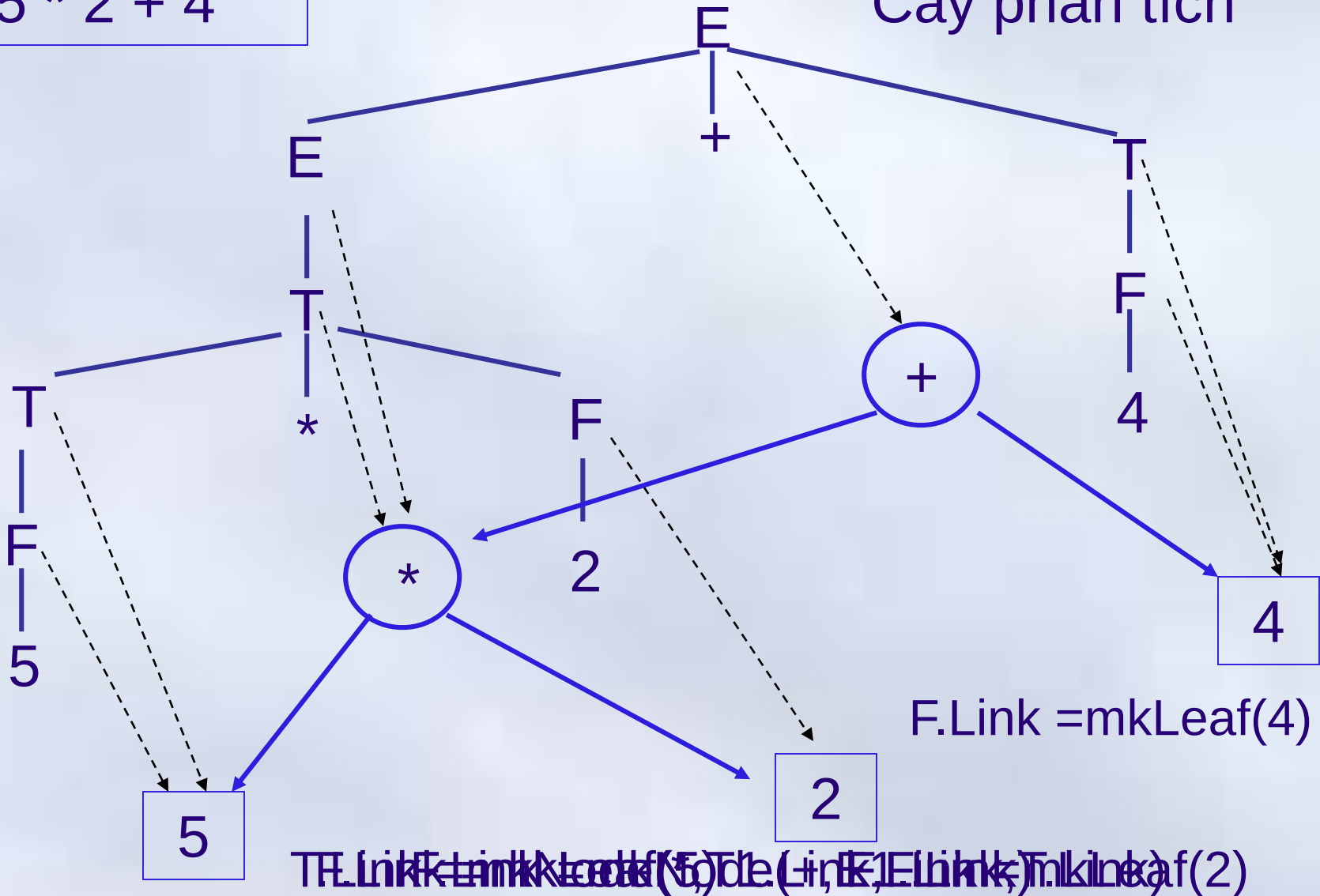
Cây cú pháp được xây dựng từ dưới lên trên

- Sau khi phân tích xong một sản xuất mới gọi luật ngữ nghĩa tương ứng (*duyet thứ tự sau*)

Xây dựng cây cú pháp → Ví dụ

5 * 2 + 4

Cây phân tích



Nội dung

- Chương trình dịch định hướng cú pháp
- Cây cú pháp
- Ký pháp Ba lan sau
- Mã 3 địa chỉ
 - Các dạng mã
 - Dịch trực tiếp cú pháp thành mã 3 địa chỉ
 - Sinh mã cho khai báo
 - Sinh mã cho lệnh gán
 - Sinh mã cho các biểu thức logic
 - Sinh mã cho các cấu trúc lập trình

Ký pháp Ba lan sau (Reverse Polish notation)

Ký pháp Ba lan: Là một ký hiệu toán học trong đó dấu đặt trước/ sau toán hạng

- Ký pháp thông thường (giữa): $5 + 6$
- Ký pháp Ba lan trước: $+ 5 6$
- Ký pháp Ba lan sau (ngược): $5 6 +$

Mục đích:

- Giảm thiểu bộ nhớ và dùng stack để tính toán

Sử dụng:

- Trong một số loại máy tính tay

Quy tắc dịch dạng trung tố $\rightarrow$ dạng hậu tố

Sản xuất	Ký pháp hậu tố	Ký pháp tiền tố
$E \rightarrow E+T$	$E \rightarrow E T +$	$E \rightarrow + E T$
$E \rightarrow T$	$E \rightarrow T$	$E \rightarrow T$
$T \rightarrow T * F$	$T \rightarrow T F *$	$T \rightarrow * T F$
$T \rightarrow F$	$T \rightarrow F$	$T \rightarrow F$
$F \rightarrow (E)$	$F \rightarrow E$	$F \rightarrow E$
$F \rightarrow \text{digit}$	$F \rightarrow \text{digit}$	$F \rightarrow \text{digit}$

Ký pháp Ba lan sau $\rightarrow$ Ví dụ 1

$$a+b*c+d$$

$E \Rightarrow E + T$	$\Leftrightarrow$	$E \Rightarrow E T +$
$\Rightarrow E + T + T$	$\Leftrightarrow$	$\Rightarrow E T + T +$
$\Rightarrow T + T + T$	$\Leftrightarrow$	$\Rightarrow T T + T +$
$\Rightarrow F + T + T$	$\Leftrightarrow$	$\Rightarrow F T + T +$
$\Rightarrow a + T + T$	$\Leftrightarrow$	$\Rightarrow a T + T +$
$\Rightarrow a + T * F + T$	$\Leftrightarrow$	$\Rightarrow a T F * + T +$
$\Rightarrow a + F * F + T$	$\Leftrightarrow$	$\Rightarrow a F F * + T +$
$\Rightarrow a + b * F + T$	$\Leftrightarrow$	$\Rightarrow a b F * + T +$
$\Rightarrow a + b * c + T$	$\Leftrightarrow$	$\Rightarrow a b c * + T +$
$\Rightarrow a + b * c + F$	$\Leftrightarrow$	$\Rightarrow a b c * + F +$
$\Rightarrow a + b * c + d$	$\Leftrightarrow$	$\Rightarrow a b c * + d +$

Ký pháp Ba lan sau $\rightarrow$ Ví dụ 2

$$(a+b) * (c+d)$$

$E \Rightarrow T$	$\Leftrightarrow$	$E \Rightarrow T$
$\Rightarrow T * F$	$\Leftrightarrow$	$\Rightarrow T F *$
$\Rightarrow F * F$	$\Leftrightarrow$	$\Rightarrow F F *$
$\Rightarrow (E) * F$	$\Leftrightarrow$	$\Rightarrow E F *$
$\Rightarrow (T + F) * F$	$\Leftrightarrow$	$\Rightarrow T F + F *$
$\Rightarrow (F + F) * F$	$\Leftrightarrow$	$\Rightarrow F F + F *$
$\Rightarrow (a + F) * F$	$\Leftrightarrow$	$\Rightarrow a F + F *$
$\Rightarrow (a + b) * F$	$\Leftrightarrow$	$\Rightarrow a b + F *$
$\Rightarrow (a + b) * (E)$	$\Leftrightarrow$	$\Rightarrow a b + E *$
$\Rightarrow (a + b) * (T + F)$	$\Leftrightarrow$	$\Rightarrow a b + T F + *$
$\Rightarrow (a + b) * (F + F)$	$\Leftrightarrow$	$\Rightarrow a b + F F + *$
$\Rightarrow (a + b) * (c + F)$	$\Leftrightarrow$	$\Rightarrow a b + c F + *$
$\Rightarrow (a + b) * (c + d)$	$\Leftrightarrow$	$\Rightarrow a b + c d + *$

Nội dung

- Chương trình dịch định hướng cú pháp
- Cây cú pháp
- Ký pháp Ba lan sau
- Mã 3 địa chỉ
 - Các dạng mã
 - Dịch trực tiếp cú pháp thành mã 3 địa chỉ
 - Sinh mã cho khai báo
 - Sinh mã cho lệnh gán
 - Sinh mã cho các biểu thức logic
 - Sinh mã cho các cấu trúc lập trình

Mã 3 địa chỉ

- Là loại mã trung gian thường dùng, tương tự mã assembly
- Chương trình trung gian là một dãy các lệnh thuộc kiểu mã 3 địa chỉ
 - Mỗi lệnh gồm tối đa 3 toán hạng
 - Tồn tại nhiều nhất một toán tử ở vế phải cộng thêm một toán tử gán
- x,y,z là các địa chỉ, tức là tên, hằng hay các tên trung gian do trình biên dịch sinh ra
 - Tên trung gian phải được sinh để thực hiện các phép toán trung gian
 - Các địa chỉ được thực hiện như con trỏ tới phần tử tương ứng của nó trong bảng ký hiệu

Mã 3 địa chỉ → Ví dụ

- Câu lệnh
 - $A = x + y * z$
- Chuyển thành mã 3 địa chỉ

$$T = y * z$$

$$A = x + T$$

T là tên trung gian

- Được bộ sinh mã trung gian sinh ra cho các toán tử trung gian

Mã 3 địa chỉ $\rightarrow$ Các dạng phổ biến

- Mã 3 địa chỉ tương tự mã Assembly:
 - Lệnh có thể có nhãn,
 - Tồn tại những lệnh chuyển điều khiển cho các cấu trúc lập trình.
- Các dạng lệnh
 - Lệnh gán $x := y \text{ op } z$.
 - Lệnh gán với phép toán 1 ngôi : $x := \text{op } y$.
 - Lệnh sao chép: $x := y$.
 - Lệnh gán có chỉ số $X := y[i]$ hoặc $x[i] = y$

Mã 3 địa chỉ → Các dạng phổ biến

➤ *Lệnh gán địa chỉ và con trỏ*

$x = \&y; \quad x = *y; \quad *x = y$

➤ *Lệnh nhảy không điều kiện: **goto L**,*

– L là nhãn của một lệnh

➤ *Lệnh nhảy có điều kiện **IF x relop y goto L**.*

– *Nếu thỏa mãn quan hệ relop ($>, \geq, <, \dots$) thì thực hiện lệnh tại nhãn L,*

– *Nếu không thỏa mãn, thực hiện câu lệnh ngay tiếp theo lệnh **IF***

Mã 3 địa chỉ → Các dạng phổ biến

➤ Gọi thủ tục với n tham số **call p, n** .

Khai báo tham số

param x

Trả về giá trị

return y

Thường dung với chuỗi lệnh 3 địa chỉ

– Lời gọi chương trình con **Call $p(X_1, X_2, \dots, x_n)$** sinh ra

param x_1

param x_2

param x_n

Call p, n

Dịch trực tiếp cú pháp thành mã 3 địa chỉ

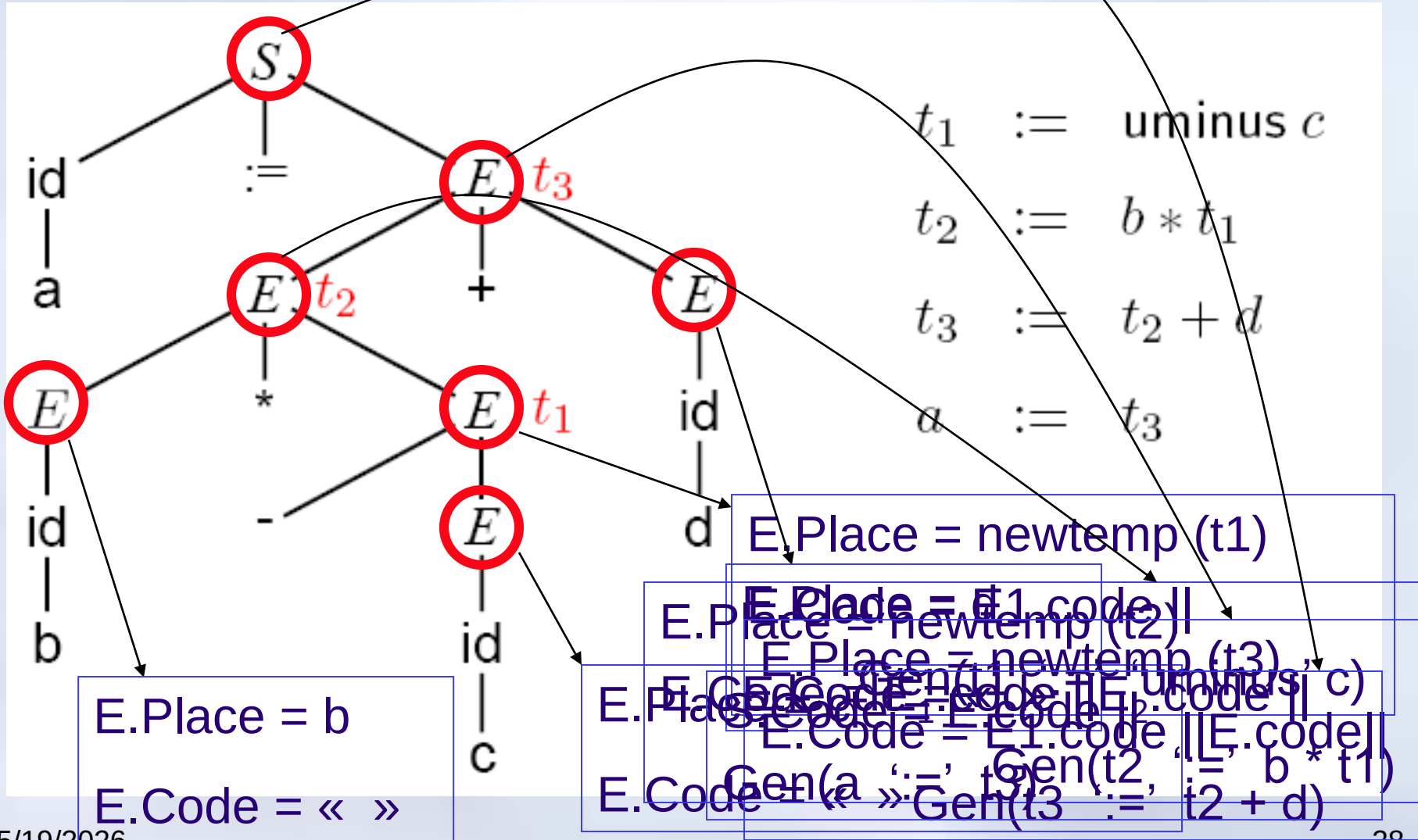
- Thuộc tính tổng hợp **S.code** biểu diễn mã ba địa chỉ của lệnh **S**
- Các tên trung gian được sinh ra cho các tính toán trung gian
- Các ký hiệu không kết thúc E có 2 thuộc tính
 - **E.place**: Thuộc tính địa chỉ/tên chứa giá trị của ký hiệu E
 - **E.code**: Chứa chuỗi mã lệnh địa chỉ để đánh giá E
- Hàm **newtemp()** sinh ra các tên trung gian t1, t2, . .
- Sử dụng hàm **gen(x ':=' y '+' z)** thể hiện mã 3 địa chỉ câu lệnh $x := y + z$
 - Các biểu thức ở các vị trí của x, y, z được đánh giá khi truyền vào hàm gen()

Dịch trực tiếp cú pháp thành mã 3 địa chỉ

Sản xuất	Quy tắc ngữ nghĩa
$S \rightarrow Id := E$	$S.Code = E.code \parallel gen(id.place \text{ ':=' } E.place)$
$E \rightarrow E_1 + E_2$	$E.Place = newTemp()$ $E.Code = E_1.code \parallel E_2.code \parallel$ $gen(E.place \text{ ':=' } E_1.place \text{ '+' } E_2.place)$
$E \rightarrow E_1 * E_2$	$E.Place = newTemp()$ $E.Code = E_1.code \parallel E_2.code \parallel$ $gen(E.place \text{ ':=' } E_1.place \text{ '*' } E_2.place)$
$E \rightarrow -E_1$	$E.place = newtemp();$ $E.code = E_1.code \parallel$ $gen(E.place \text{ ':=' 'uminus' } E_1.place)$
$E \rightarrow (E)$	$E.place = E_1.place ; E.code = E_1.code$
$E \rightarrow Id$	$E.place = id.place ; E.code = ''$

Dịch trực tiếp cú pháp thành mã 3 địa chỉ

Câu lệnh gán: $a := b * -c + d$



Dịch trực tiếp cú pháp thành mã 3 địa chỉ

Ví dụ: Câu lệnh lặp while

Sản xuất	Quy tắc ngữ nghĩa
$S \rightarrow \text{while } E \text{ do } S1$ S.Begin E.code If E.place = 0 goto S.After S1.code Goto S.begin S.After	S.Begin = newLabel() S.After = newLabel() S.Code = /*Sinh mã cho lệnh while gồm*/ gen(S.begin ':') E.code /*Sinh mã cho lệnh đánh giá E*/ Gen('if' E.place '=' 0 'goto' S.After) S1.code /*Sinh mã cho lệnh S1*/ gen('goto' S.Begin) /*Sinh mã cho goto*/ Gen(S.After ':') /*Sinh mã cho nhãn mới*/
Hàm newLabel(): Sinh ra một nhãn mới	

Ví dụ:

While $a + b$ Do $a := a * b$

```
S.Begin =  
    newLabel() =  
    L1  
S.After =  
    newLabel() =  
    L2
```

S.Code =

```
L1:  t1 := a + b  
      If t1 = 0 goto L2  
      t2 = a * b  
      a := t2  
      goto L1  
  
L2:
```

Cài đặt lệnh 3 địa chỉ → Biểu diễn bộ bốn

- Sử dụng cấu trúc gồm 4 trường: **Op**, **Arg1**, **Arg2**, **Result**
 - **Op**: Chứa mã nội bộ của toán tử
 - Các trường **Arg1**, **Arg2**, **Result** trỏ tới các ô trong bảng ký hiệu, ứng với các tên tương ứng
- Câu lệnh dạng **a := b Op c**
 - Đặt b vào Arg1, c vào Arg2 và a vào Result
- Câu lệnh một ngôi: **a := b; a := -b**
 - Không sử dụng Arg2

Cài đặt lệnh 3 địa chỉ→Biểu diễn bộ bốn

Ví dụ lệnh $a = -b * (c+d)$

Lệnh 3 địa chỉ

$t1 := -b$

$t2 := c + d;$

$t3 := t1 * t2;$

$a := t3$

Biểu diễn bởi dãy các bộ 4

	Op	Arg1	Arg2	Result
0	uminus	b		t1
1	+	c	d	t2
2	*	t1	t2	t3
3	:=	t3		a

Các tên tạm phải được đưa vào bảng ký hiệu

Cài đặt lệnh 3 địa chỉ → Biểu diễn bộ ba

- Mục đích để tránh đưa tên tạm vào bảng ký hiệu
- Tham khảo tới giá trị tạm thời bằng vị trí lệnh sử dụng tính ra giá trị này
- Bỏ trường **Result**, Các trường **Arg1**, **Arg2** trỏ tới phần tử tương ứng trong bảng ký hiệu hoặc câu lệnh tương ứng

	Op	Arg1	Arg2
0	uminus	b	
1	+	c	d
2	*	(0)	(2)
3	:=	a	(2)

Sinh mã cho khai báo

- Sử dụng biến toàn cục offset
 - Trước khi bắt đầu khai báo: $\text{offset} = 0$
 - Với mỗi khai báo biến sẽ đưa tên đối tượng, kiểu và giá trị của offset vào bảng ký hiệu
 - Tăng offset lên bằng kích thước của dữ liệu
- Các tên trong chương trình con được truy xuất thông qua địa chỉ tương đối offset
 - Khi gặp tên đối tượng (biến), dựa vào trường offset để biết vị trí trong vùng dữ liệu

Sinh mã cho khai báo

Sản xuất	Quy tắc ngữ nghĩa
$P \rightarrow MD$	$\{\}$
$M \rightarrow \varepsilon$	$\{Offset = 0\}$
$D \rightarrow D ; D$	
$D \rightarrow Id : T$	$enter(id.name, T.type, offset)$ $Offset = Offset + T.Width$
$T \rightarrow \text{interger}$	$T.type = \text{Interger}; T.width = 2$
$T \rightarrow \text{real}$	$T.type = \text{real}; T.width = 4$
$T \rightarrow \text{array}[num] \text{ of } T_1$	$T.type = \text{array}(1..num.val, T_1.type)$ $T.width = num.val * T_1.width$
Hàm Enter(name, type, offset) thêm một đối tượng vào bảng ký hiệu với tên (name), kiểu(type) và địa chỉ tương đối (offset) của vùng dữ liệu của nó.	

Sinh mã cho khai báo → Ví dụ

```
A: Integer;  
B: Integer;  
C: Integer;  
  
B := 10  
C := 20  
A := B * C
```

C	Integer	4
B	Integer	2
A	Integer	0
SymbolTable		

20
10

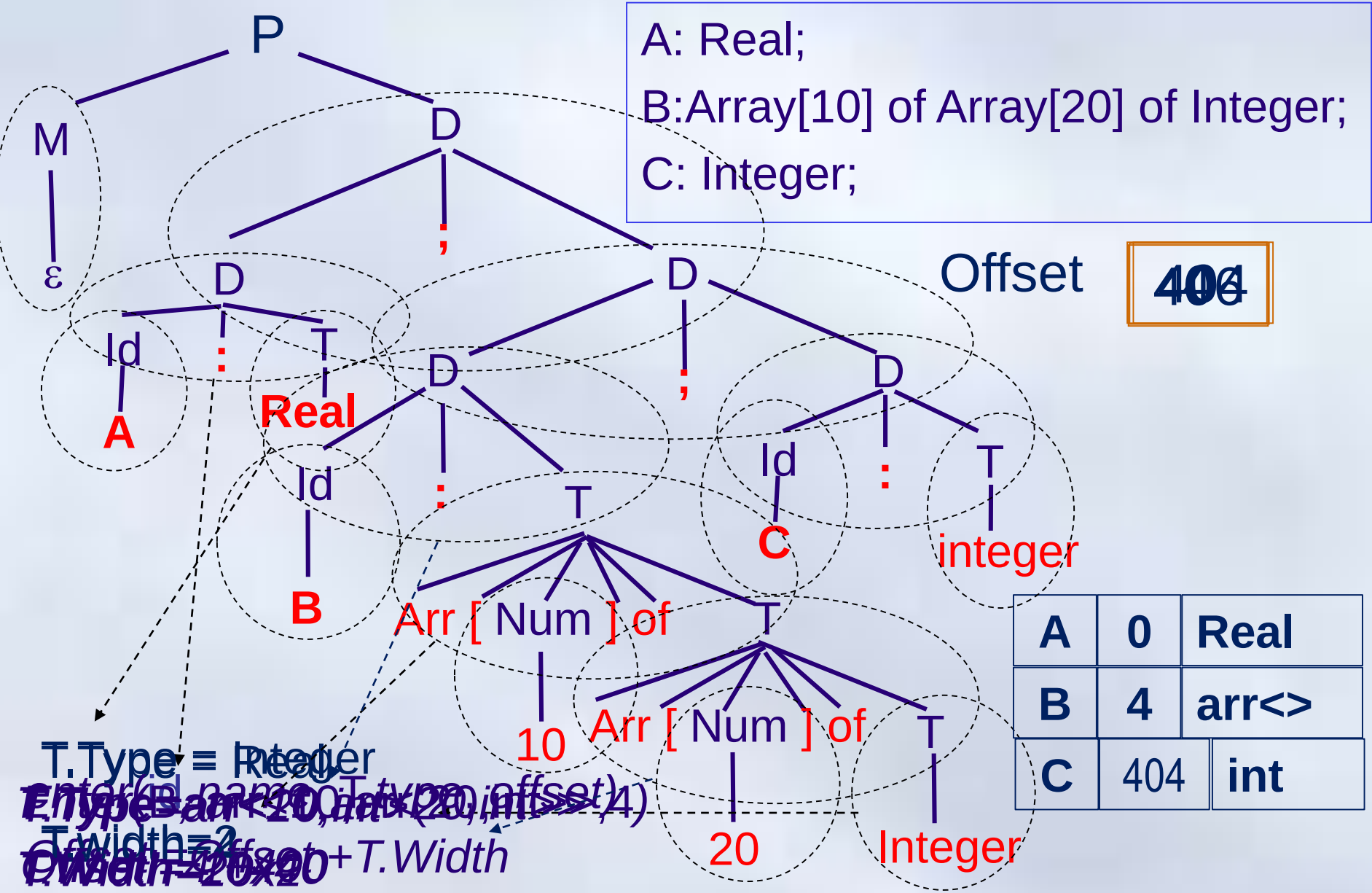
[2] := 10
[4] := 20
[0] := [2] * [4]

A := A * B + C ?

Thêm biến tạm T vào bảng ký hiệu, T có offset là : 6

Code Segment

Sinh mã cho khai báo → Ví dụ



Lưu trữ thông tin về phạm vi

- Văn phạm cho phép các chương trình con bao nhau
 - Khi bắt đầu phân tích chương trình con, phần khai báo của chương trình bao tạm dừng
 - Dùng một bảng ký hiệu riêng cho mỗi chương trình con
- Văn phạm của khai báo này:

$P \rightarrow D$

$D \rightarrow D; D \mid id : T \mid \text{proc } id ; D ; S$

- Khi khai báo chương trình con $D \rightarrow \text{proc } id \text{ } D1; S$ được phân tích thì các khai báo trong $D1$ được lưu trong bảng ký hiệu mới.

Lưu trữ thông tin về phạm vi → Ví dụ

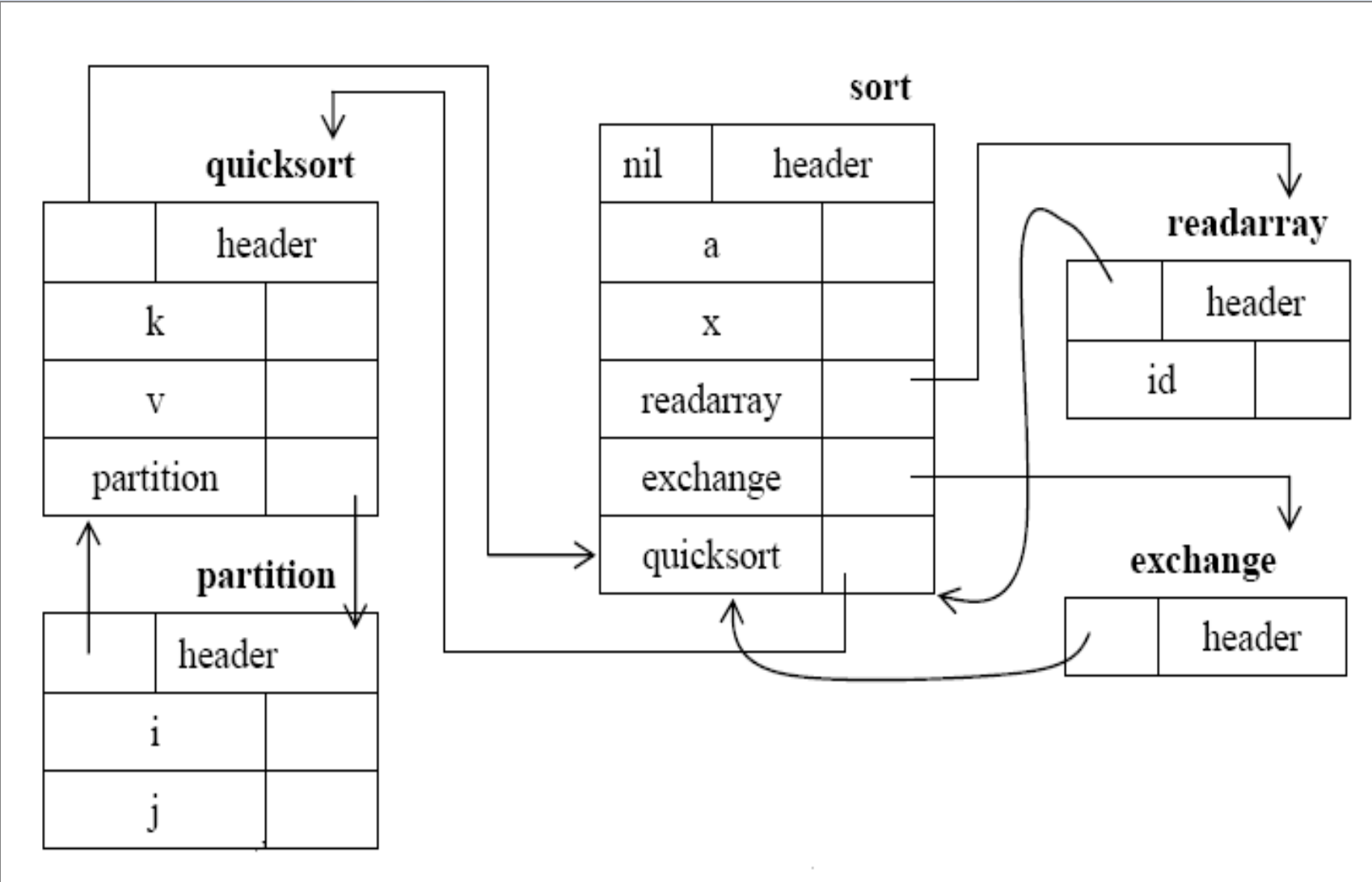
```

1) Program sort;    //Chương trình Quicksort
2) Var a: array[0..10] of integer;
3)    x: integer;
4)    Procedure readarray;
5)        Var i: integer;
6)        Begin ..... end {readarray};
7)    Procedure exchange(i, j: integer);
8)        Begin {exchange} end;
9)    Procedure quicksort(m, n: integer);
10)        Var k, v: integer;
11)    Function partition(y,z: integer): integer;
12)        Var i, j : integer;
13)        Begin .....exchange(i,j) end; {partition}
14)    Begin ... end; {quicksort}
15) Begin ... end; {sort}

```

Lưu trữ thông tin về phạm vi→Ví dụ

- Các bảng ký hiệu của chương trình sort



Quy tắc ngữ nghĩa → Các thao tác

- **mktable(previous)** – tạo một bảng kí hiệu mới, bảng này có *previous* chỉ đến bảng cha của nó.
- **enter(table,name,type,offset)** – thêm một đối tượng mới có tên *name* vào bảng kí hiệu được chỉ ra bởi *table* và đặt kiểu là *type* và địa chỉ tương đối là *offset* vào các trường tương ứng.
- **enterproc(table,name,newbtable)** – tạo một phần tử mới trong bảng *table* cho chương trình con *name*, *newbtable* trỏ tới bảng kí hiệu của CTC này.
- **addwidth(table,width)** – ghi tổng kích thước của tất cả các p/tử trong bảng kí hiệu vào header của bảng.

Khai báo trong chương trình con

Sản xuất	Quy tắc ngữ nghĩa
$P \rightarrow MD$	<i>addwidth(top(tblptr), top(offset)); pop(tblptr); pop(offset)</i>
$M \rightarrow \varepsilon$	<i>t:=mktable(null); push(t, tblptr); push(0, offset)</i>
$D \rightarrow D ; D$	
$D \rightarrow \text{proc id}; ND_1; S$	<i>t:=top(tblpr); addwidth(t, top(offset)); pop(tblptr); pop(offset); enterproc(top(tblptr), id.name, t)</i>
$N \rightarrow \varepsilon$	<i>t:=mktable(top(tblptr)); push(t, tblptr); push(0, offset);</i>
$D \rightarrow id : T$	<i>enter(top(tblptr), id.name, T.type, top(offset); top(offset):=top(offset) + T.width</i>

Tblptr: là Stack dùng chứa các con trỏ trỏ tới bảng ký hiệu

Offset: Là Stack dùng lưu trữ các Offset

Xử lý các khai báo trong chương trình con

- Sản xuất: $P \rightarrow MD$:
 - Hoạt động của cây con M được thực hiện trước
- Sản xuất: $M \rightarrow \varepsilon$:
 - Tạo bảng ký hiệu cho phạm vi ngoài cùng (*chương trình sort*) bằng lệnh ***mktable(nil)*** // *Không có SymTab cha*
 - Khởi tạo stack ***tblptr*** với bảng ký hiệu vừa tạo ra
 - Đặt offset = 0.
- Sản xuất: $N \rightarrow \varepsilon$:
 - Tạo ra một bảng mới ***mktable(top(tblptr))***
 - Tham số ***top(tblptr)*** cho giá trị con trỏ tới bảng cha
 - Thêm bảng mới vào đỉnh stack ***tblptr*** // *push(t, tblptr)*
 - 0 được đẩy vào stack ***offset*** // *push(0, Offset)*

N đóng vai trò tương tự M khi một khai báo CTC xuất hiện

Xử lý các khai báo trong chương trình con

- Với mỗi khai báo **id: T**
 - một phần tử mới được tạo ra cho **id** trong bảng kí hiệu hiện hành (*top(tblptr)*)
 - Stack *tblptr* không đổi,
 - Giá trị **top(offset)** được tăng lên bởi **T.width**.
- Khi **D → proc id ; N D₁ ; S** diễn ra
 - Kích thước của tất cả các đối tượng dữ liệu khai báo trong **D₁** sẽ nằm trên đỉnh stack **offset**.
 - Kích thước này được lưu trữ bằng cách dùng **Addwidth()**,
 - Các stack *tblptr* và **offset** bị lấy mất phần tử trên cùng (**pop()**)
 - Thao tác thực hiện trên các khai báo của chương trình con.

Ví dụ

```

X:int;
M:Array[10] of int;
Proc A;
    X:int;
    N : int;
    Proc B;
        X:int;
        SB
    SA
S

```

- Xây dựng cây phân tích có chú giải cho chương trình bên.
- Xây dựng bảng ký hiệu, có chỉ rõ vị trí các biến
- Nếu **S_B** là lệnh **M[4] + N * X;**
Cho biết địa chỉ của các đối tượng trong lệnh trên
- Nếu **S_B** là câu lệnh **CALL A;**
 - Lệnh thực hiện được không? Hãy giải thích
 - Cần sửa lại như thế nào để lệnh thực hiện được như bình thường.

Sinh mã cho lệnh gán → Các hàm

- Hàm lookup()
 - Tìm trong bảng kí hiệu xem một tên (*id.name*) đã tồn tại
 - Tìm trong bảng ký hiệu hiện thời (*top(tblptr)*)
 - Nếu không có, tìm trong các bảng ký mức cha (*con trỏ trong phần header của bảng ký hiệu*)
 - Nếu tồn tại, trả về con trỏ tới vị trí; ngược lại, trả về nil.
 - **Vị trí Id:** Vị trí trong bảng và độ lệch mức
- Thủ tục emit()
 - Ghi mã 3 địa chỉ vào một tập tin output
 - gen() xây dựng thuộc tính *code* cho các kí hiệu chưa kết thúc
 - Khi thuộc tính code của kí hiệu không kết thúc trong vế trái sản xuất được tạo ra bằng cách nối thuộc tính code của kí hiệu không kết thúc trong vế phải theo đúng thứ tự xuất hiện, sẽ ghi ra tập tin bên ngoài

Sinh mã cho lệnh gán

Sản xuất	Quy tắc ngữ nghĩa
$S \rightarrow \text{Id} := E$	$p := \text{lookup}(\text{id.name})$ if $p \neq \text{nil}$ then emit($p := E.\text{place}$) else error()
$E \rightarrow E_1 + E_2$	$E.\text{Place} = \text{newTemp}()$ emit($E.\text{place} := E_1.\text{place} + E_2.\text{place}$)
$E \rightarrow E_1 * E_2$	$E.\text{Place} = \text{newTemp}()$ emit($E.\text{place} := E_1.\text{place} * E_2.\text{place}$)
$E \rightarrow -E_1$	$E.\text{place} = \text{newtemp}();$ emit($E.\text{place} := \text{'uminus'} E_1.\text{place}$)
$E \rightarrow (E)$	$E.\text{place} = E_1.\text{place} ;$
$E \rightarrow \text{Id}$	$p := \text{lookup}(\text{id.name})$ if $p \neq \text{nil}$ then $E.\text{place} := p$ else error()

Địa chỉ hóa các phần tử của mảng

- Các phần tử của mảng được lưu trữ trong một khối ô nhớ kế tiếp nhau để truy xuất nhanh
- **Mảng một chiều:** nếu kích thước một phần tử là w
 $\Rightarrow$ địa chỉ tương đối phần tử thứ i của mảng A là

$$A[i] = \text{base} + (i - \text{low}) * w$$

$$A[i] = i * w + (\text{base} - \text{low} * w)$$

- *Low*: cận dưới tập chỉ số. Một số ngôn ngữ, $\text{low} = 0 // 1$
- *Base*: địa chỉ tương đối của vùng nhớ cấp phát cho mảng (địa chỉ tương đối của phần tử $A[\text{low}]$)
- $c = \text{base} - \text{low} * w$ có thể được tính tại thời gian dịch và lưu trong bảng kí hiệu. Vậy $A[i] = i * w + c$

- **Mảng 2 chiều:** mảng của mảng 1 chiều

Sinh mã cho biểu thức logic

- Biểu thức logic được sinh bởi văn phạm sau:
$$E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{not } E$$
$$\mid (E) \mid \text{id relop id} \mid \text{true} \mid \text{false}$$
- Trong đó:
 - **Or** và **And** kết hợp trái
 - **Or** có độ ưu tiên thấp nhất tiếp theo là **And**, và **Not** (*Văn phạm trên nhập nhằng*)
- Mã hóa giá trị logic true/false
 - Mã hóa bằng số; đánh giá một biểu thức logic như một biểu thức số học

Sinh mã cho biểu thức logic → Ví dụ

- Biểu thức **a or b and not c**
 - Mã 3 địa chỉ:
t1 = not c
t2 = b and t1
t3 = a or t2
- Biểu thức **a < b**
 - Tương đương lệnh **if a < b then 1 else 0.**
 - Mã 3 địa chỉ tương ứng (*g/thiết lệnh bắt đầu 100*)
100: if a < b goto 103
101: t := 0
102: goto 104
103: t := 1
104:

Sinh mã cho biểu thức logic: biểu diễn số

Sản xuất	Quy tắc ngữ nghĩa
$E \rightarrow E_1 \text{ or } E_2$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(E.\text{place} \text{ ':=' } E1.\text{place} \text{ 'or' } E2.\text{place})$
$E \rightarrow E_1 \text{ and } E_2$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(E.\text{place} \text{ ':=' } E1.\text{place} \text{ 'and' } E2.\text{place})$
$E \rightarrow \text{not } E_1$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(E.\text{place} \text{ ':=' 'not' } E1.\text{place})$
$E \rightarrow \text{Id}_1 \text{ relop } \text{Id}_2$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(\text{'if' id1.place relop id2.place 'goto' nextstat+3'})$ $\text{Emit}(E.\text{place} \text{ ':=' '0'}); \text{Emit}(\text{'goto' nextstat+2});$ $\text{Emit}(E.\text{place} \text{ ':=' '1'});$
$E \rightarrow \text{True}$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(E.\text{place} \text{ ':=' '1'})$
$E \rightarrow \text{False}$	$E.\text{Place} = \text{newTemp}();$ $\text{Emit}(E.\text{place} \text{ ':=' '0'})$

Nextstat (*next statement*) cho biết chỉ số của câu lệnh 3 địa chỉ tiếp theo

Sinh mã cho biểu thức logic → Ví dụ

- Biểu thức $a < b \text{ AND } c > d$
 - $E \rightarrow E$ and $E \rightarrow Id < Id$ and $E \rightarrow Id < Id$ and $Id > Id$

100: if a < b goto 103

101: t1 := 0

102: goto 104

103: t1 := 1

104: if c > d goto 107

105: t2 := 0

106: goto 108

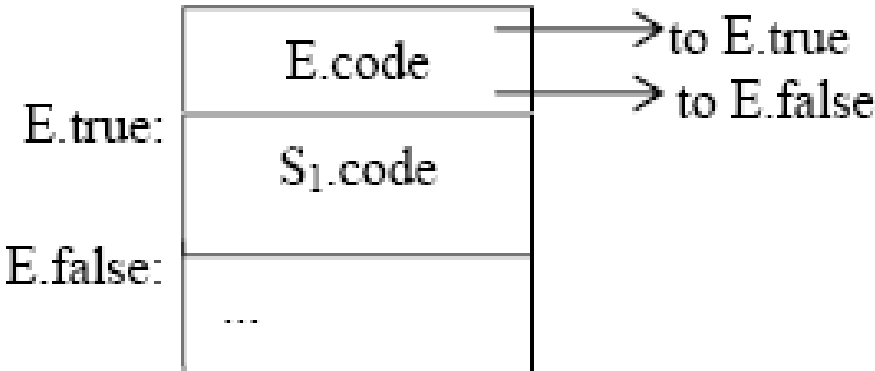
107: t2 := 1;

108: t3 := t1 and t2

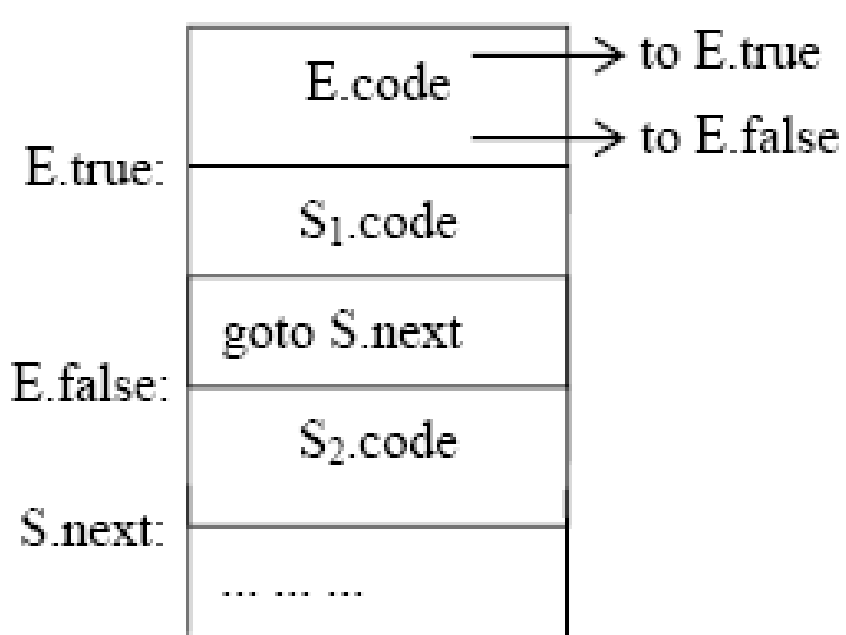
Sinh mã cho các cấu trúc lập trình

- Cấu trúc: $S \rightarrow \text{if } E \text{ then } S_1 \mid$
 $\text{if } E \text{ then } S_1 \text{ else } S_2 \mid$
 $\text{while } E \text{ do } S_1$
- **E** là biểu thức logic. E có 2 nhãn thuộc tính
 - **E.true**: Thuộc tính chứa nhãn của dòng điều khiển nếu biểu thức **E** là **true**
 - **E.false**: Thuộc tính chứa nhãn của dòng điều khiển nếu biểu thức **E** là **false**
- **S.code**: Dãy lệnh 3 địa chỉ được sinh ra bởi **S**
- **S.next**: Th.tính chứa nhãn địa chỉ của lệnh 3 địa chỉ đầu tiên sẽ thực hiện sau dãy lệnh của cấu trúc S
- **S.begin**: nhãn địa chỉ của lệnh đầu tiên được sinh ra cho cấu trúc S

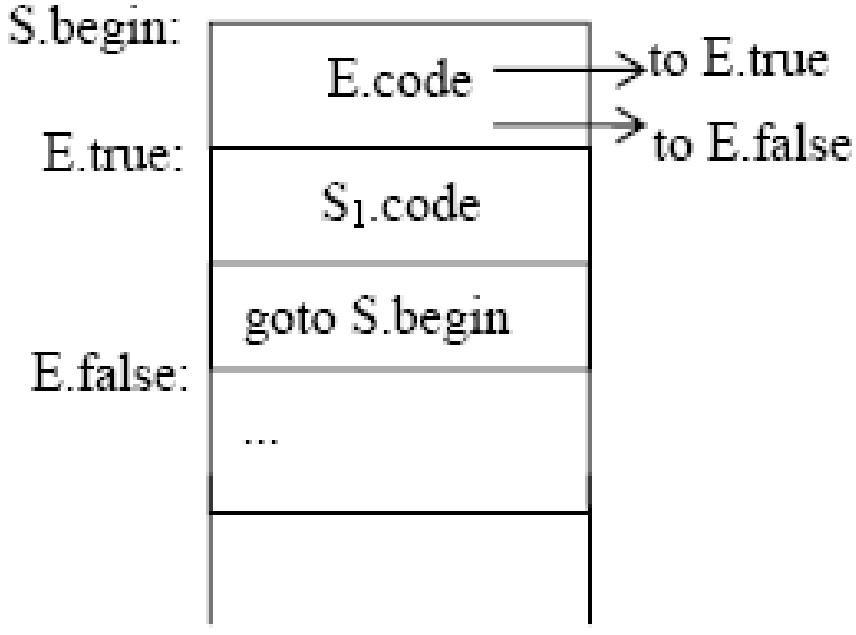
Sinh mã cho các cấu trúc lập trình



(a) if -then



(b) if -then-else



(c) while-do

Dịch trực tiếp cú pháp cho các cấu trúc lập trình

Sản xuất	Quy tắc ngữ nghĩa
$S \rightarrow \text{if } E \text{ then } S_1$	$E.\text{True} = \text{newLabel}();$ $E.\text{False} = S.\text{next}; \quad S_1.\text{next} = S.\text{next}$ $S.\text{Code} = E.\text{code} \parallel \text{gen}(E.\text{true} \text{ ' : ' }) \parallel S_1.\text{code}$
$S \rightarrow \text{if } E \text{ then } S_1$ $\text{else } S_2$	$E.\text{True} = \text{newLabel}(); \quad E.\text{False} = \text{newLabel}();$ $S_1.\text{next} = S.\text{next}; \quad S_2.\text{next} = S.\text{next}$ $S.\text{Code} = E.\text{code} \parallel \text{gen}(E.\text{true} \text{ ' : ' }) \parallel S_1.\text{code} \parallel$ $\text{gen}(\text{'goto' } S.\text{next}) \parallel$ $\text{gen}(E.\text{false} \text{ ' : ' }) \parallel S_2.\text{code}$
$S \rightarrow \text{while } E \text{ do } S_1$	$S.\text{Begin} = \text{newLabel}(); \quad E.\text{True} = \text{newLabel}();$ $E.\text{False} = S.\text{next}; \quad S_1.\text{next} = S.\text{Begin}$ $S.\text{Code} = \text{gen}(S.\text{begin} \text{ ' : ' }) \parallel E.\text{code} \parallel \text{gen}(E.\text{true} \text{ ' : ' })$ $\parallel S_1.\text{code} \parallel \text{gen}(\text{'goto' } S.\text{Begin});$

Sinh mã cho biểu thức logic trong cấu trúc lập trình

- Nếu E có dạng: **$a < b$**
 - Mã lệnh sinh ra có dạng
If $a < b$ then goto E.true
goto E.false
- Nếu E có dạng: **$E_1 \text{ or } E_2$** thì
 - Nếu E_1 là true thì E cũng là true
 - Nếu E_1 là false thì phải đánh giá E_2 ; E sẽ là true hay false phụ thuộc E_2
- Tương tự với **$E_1 \text{ and } E_2$**
- Nếu E có dạng **$\text{not } E_1$**
 - Nếu E_1 là true, E là false và ngược lại

**E.Code sinh ra
như thế nào?**

Sinh mã cho biểu thức logic trong cấu trúc lập trình

Sản xuất	Quy tắc ngữ nghĩa
$E \rightarrow E_1 \text{ or } E_2$	$E_1.\text{true} := E.\text{true}$ $E_1.\text{false} := \text{newLabel}()$ $E_2.\text{true} := E.\text{true}$ $E_2.\text{false} := E.\text{false}$ $E.\text{Code} = E_1.\text{code} \parallel \text{gen}(E_1.\text{false} ': ') \parallel E_2.\text{code}$
$E \rightarrow E_1 \text{ and } E_2$	$E_1.\text{true} := \text{newLabel}()$ $E_1.\text{false} := E.\text{false}$ $E_2.\text{true} := E.\text{true}$ $E_2.\text{false} := E.\text{false}$ $E.\text{Code} = E_1.\text{code} \parallel \text{gen}(E_1.\text{true} ': ') \parallel E_2.\text{code}$
Chú ý: E.True và E.false là các thuộc tính kế thừa	

Sinh mã cho biểu thức logic trong cấu trúc lập trình

Sản xuất	Quy tắc ngữ nghĩa
$E \rightarrow \text{not } E_1$	$E_1.\text{true} := E.\text{false}$ $E_1.\text{false} := E.\text{true}$ $E.\text{Code} = E_1.\text{code}$
$E \rightarrow (E_1)$	$E_1.\text{True} := E.\text{true}$ $E_1.\text{false} := E.\text{false}$ $E.\text{Code} := E_1.\text{code}$
$E \rightarrow \text{id}_1 \text{ relop } \text{id}_2$	$E.\text{Code} := \text{gen}(\text{'if' id1.place relop id2.place 'goto' E.true})$ $\text{gen}(\text{'goto' E.false});$
$E \rightarrow \text{True}$	$E.\text{Code} := \text{gen}(\text{'goto' E.true})$
$E \rightarrow \text{False}$	$E.\text{Code} := \text{gen}(\text{'goto' E.false})$

Sinh mã cho biểu thức logic → Ví dụ 1

$$a < b \text{ or } c < d \text{ and } e < f$$

- Giả thiết Ltrue và Lfalse là nhãn đi đến ứng với các giá trị true và false của biểu thức
- Dựa trên quy tắc ngữ nghĩa, sinh ra

if $a < b$ goto Ltrue

goto L1:

L1: if $c < d$ goto L2

goto Lfalse

L2: if $e < f$ goto Ltrue

goto Lfalse

Sinh mã cho biểu thức logic $\rightarrow$ Ví dụ 1• $E \rightarrow a < b$

$E.code = \text{if } a < b \text{ goto } E.true \text{ goto } E.false$

• $E \rightarrow E_1 \text{ or } E_2$

– $E_1.true := E.true \Rightarrow E_1.true = Ltrue$

• $Ltrue$ là nhãn đi tới nếu biểu thức là true

• $Lfalse$ là nhãn đi tới nếu biểu thức là false

– $E_1.false := L1; //E1.False = newLabel()$

– $E_2.true := E.true = Ltrue; \quad E_2.false := E.false = Lfalse$

– $E.Code = E_1.code \parallel \text{gen}(E_1.false \text{ ':' }) \parallel E_2.code$

$\text{if } a < b \text{ goto } E_1.true \text{ goto } E_1.false \parallel E_1.false \parallel E_2.code$

$\text{if } a < b \text{ goto } Ltrue \text{ goto } L1 \quad L1: \parallel E_2.code \quad (1)$

Sinh mã cho biểu thức logic $\rightarrow$ Ví dụ 1

- $E \rightarrow c < d$ $E.code = \text{if } c < d \text{ goto } E.true \text{ goto } E.false$
- $E \rightarrow e < f$ $E.code = \text{if } e < f \text{ goto } E.true \text{ goto } E.false$
- $E \rightarrow E_1 \text{ and } E_2$
 - $E_1.true := L2$
 - $E_1.false := E.false = LFalse;$
 - $E_2.true := E.true = Ltrue; \quad E_2.false := E.false = Lfalse$
 - $E.Code = E_1.code \parallel \text{gen } (E_1.true \text{ ': '}) \parallel E_2.code$
 $\text{if } c < d \text{ goto } E_1.true \text{ goto } E_1.false \parallel E_1.true : \parallel E_2.code$
 $\text{if } c < d \text{ goto } L2 \text{ goto } Lfalse \text{ } L2: \text{if } e < f \text{ goto } E_2.true \text{ goto } E_2.false$
 $\text{if } c < d \text{ goto } L2 \text{ goto } Lfalse \text{ } L2: \text{if } e < f \text{ goto } Ltrue \text{ goto } Lfalse \text{ (2)}$

Sinh mã cho biểu thức logic $\rightarrow$ Ví dụ 2**While** $a \neq b$ **do****If** $a > b$ **Then** $a := a - b \text{ // } (S_1)$ **Else** $b := b - a \text{ // } (S_2)$ $S \Rightarrow Id := E \Rightarrow Id := E_1 - E_2 \Rightarrow Id := Id - Id$ **S_1 .Code** $t1 := a - b$ $a := t1$

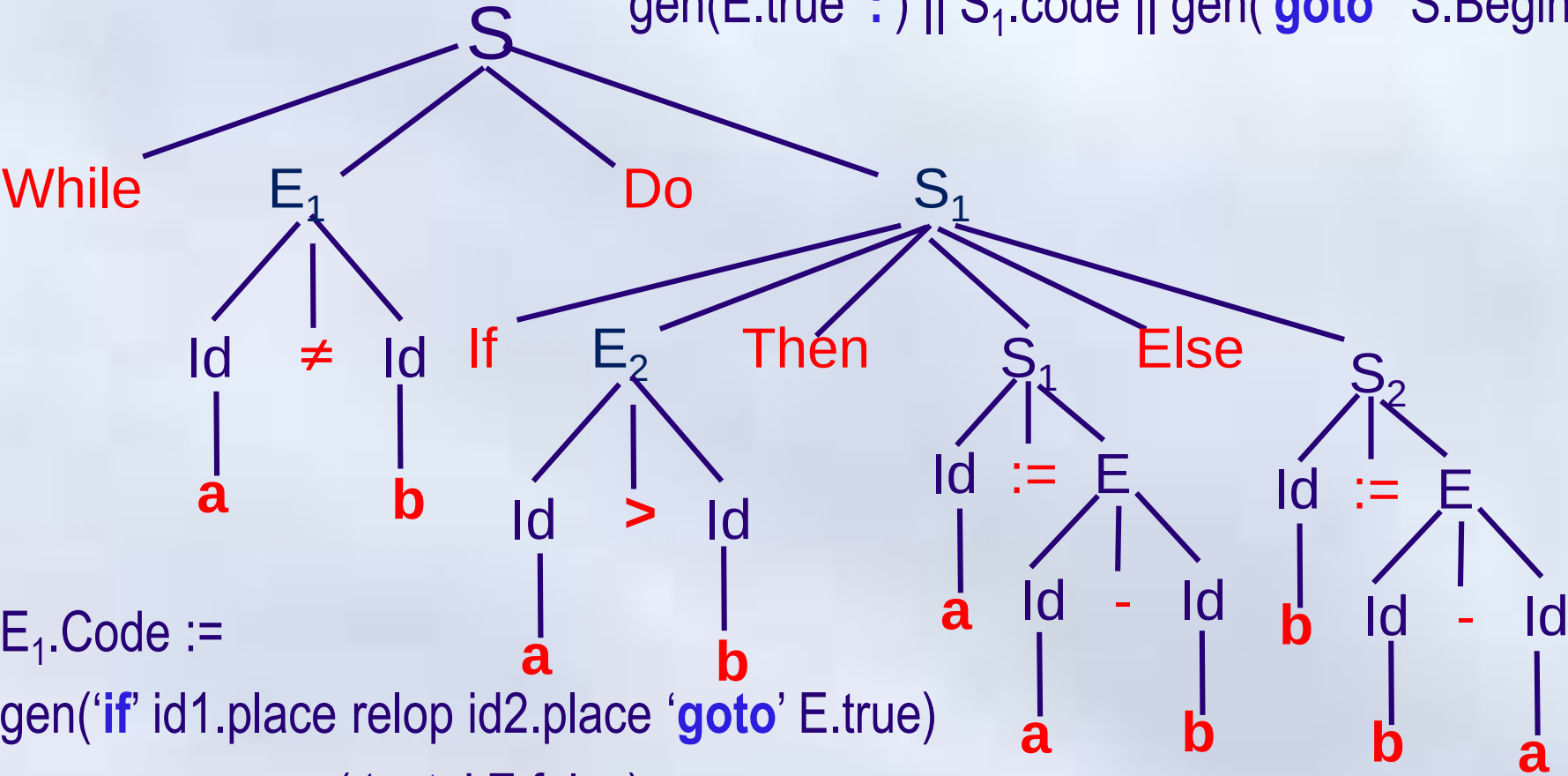
(A)

 S_2 .Code $t2 := b - a$ $a := t2$

(B)

Sinh mã cho biểu thức logic → Ví dụ 2

```
S.Begin=newLabel(); E1.True = newLabel();  
E1.False = S.next; S1.next = S.Begin  
S.Code = gen(S.begin ':') ||E.code ||  
gen(E.true ':') || S1.code || gen('goto ' S.Begin);
```



```
E1.Code :=  
gen('if' id1.place relop id2.place 'goto' E.true)  
gen( 'goto' E.false);
```

Sinh mã cho biểu thức logic → Ví dụ 2

$S \rightarrow \textbf{While } E \textbf{ do } S_1$

S.Begin = L1 // S.Begin=newLabel();

E.True = L2 // E.True = newLabel();

E.False = Next // E.False = S.next();

S₁.next = L1 // S₁.next = S.Begin

S.Code = L1: || E.code || L2 || S₁.code || goto L1 (1)

$E \rightarrow a \neq b$

E.Code := if a ≠ b goto L2 goto Next (2)

E.Code:=gen('if' id1.place relop id2.place 'goto' E.true)
gen('goto' E.false);

Sinh mã cho biểu thức logic → Ví dụ 2

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

E.True = L3	// E.True = newLabel();	(3)
E.False = L4	// E.False = newLabel();	
S_1 .next = L1	// S_1 .next = S.next;	
S_2 .next = L1	// S_2 .next = S.next	
S.Code = E.code L3 : S1.code goto L1 L4 : S2.code		

$E \rightarrow a > b$

E.Code := **if** a > b **goto** L3 **goto** L4 (4)

E.Code:=gen('if' id1.place relop id2.place 'goto' E.true)
gen('goto' E.false);

Sinh mã cho biểu thức logic → Ví dụ 2

$1 + 2 + + 4 + A + B$

L1 : **if** $a \neq b$ **goto** L2
 goto Next

L2 : **if** $a > b$ **goto** L3
 goto L4

L3 : $t1 := a - b$
 $a := t1$
 goto L1

L4: $t2 := b - a$
 $a := t2$
 goto L1

Next:

Biểu thức hỗn hợp

- Thực tế, các biểu thức logic thường chứa các biểu thức số học
 - $(a+b)<c$
- Xét văn phạm
$$E \rightarrow E + E \mid E \text{ and } E \mid E \text{ relop } E \mid \text{Id}$$

$E \text{ and } E$ đòi hỏi 2 đối số phải là logic

$E + E, E \text{ relop } E$: Các đối số là biểu thức toán học
- Để sinh mã trong trường hợp phức hợp
 - Dùng thuộc tính tổng hợp $E.\text{Type}$ cho biết kiểu là *arith* hay *logic*.

Chương 5: Sinh mã

1. Sinh mã trung gian

2. Sinh mã đích

3. Tối ưu mã

Nội dung

1. Giới thiệu
2. Môi trường thực hiện: Máy ngăn xếp
 - Bộ thông dịch cho máy ngăn xếp
3. Sinh mã đích từ mã trung gian
 - Mã trung gian là cây cú pháp
 - Mã trung gian là ký pháp Ba lan sau
 - Mã trung gian là mã 3 địa chỉ
4. Sinh mã đích từ mã nguồn
 - Xây dựng bảng ký hiệu
 - Sinh mã cho câu lệnh

1. Giới thiệu

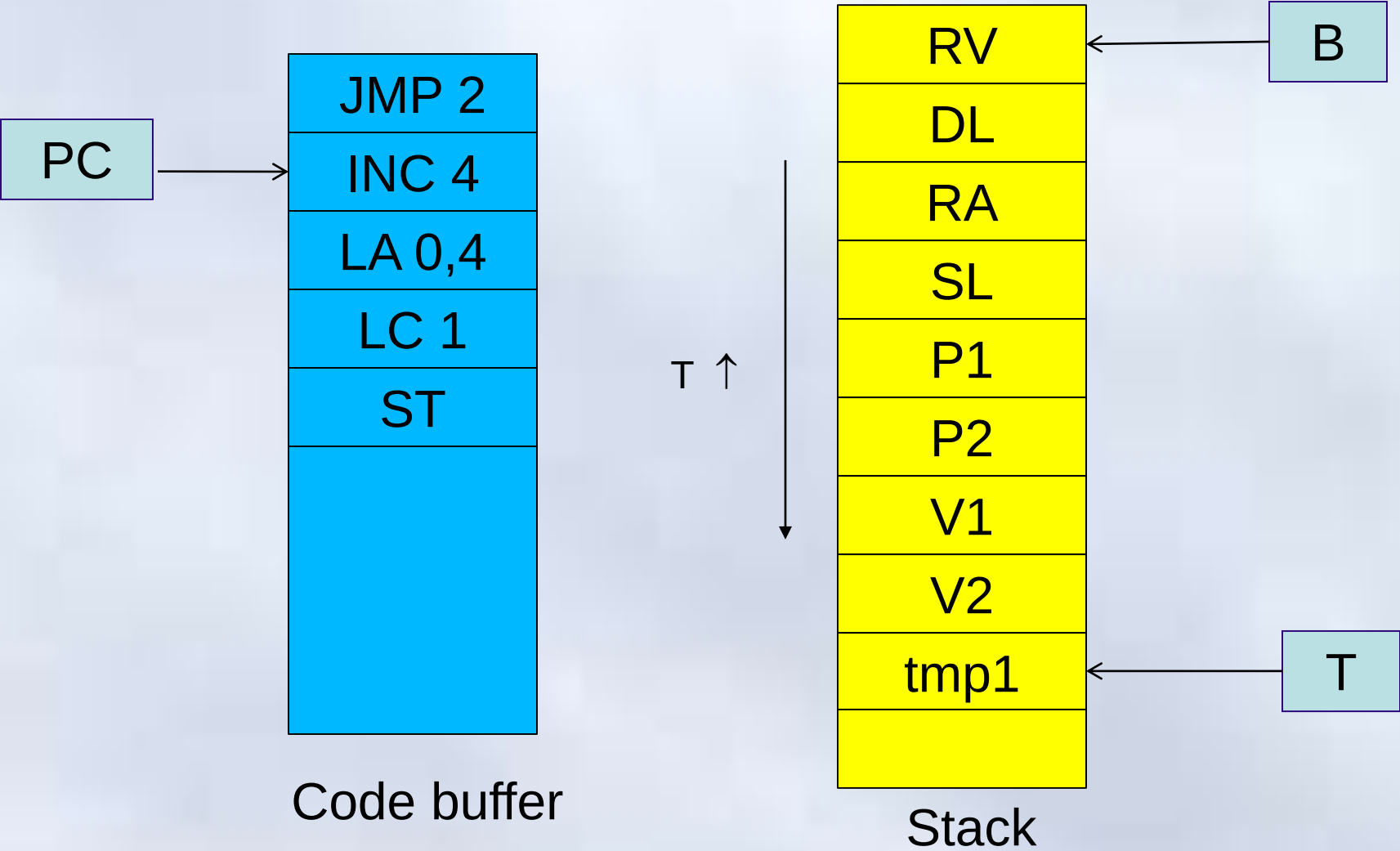
- Chương trình đích viết trên một ngôn ngữ trung gian
- Là dạng Assembly của máy giả định
 - Máy ảo (*Máy ảo làm việc với bộ nhớ stack*)
- Việc thực hiện chương trình thông qua một bộ thông dịch *interpreter*
 - *Interpreter* mô phỏng hành động của máy ảo
 - Thực hiện tập lệnh assembly của nó
- Chương trình đích được dịch từ
 - Mã trung gian
 - Mã nguồn

2. Môi trường thực hiện

- Sử dụng máy ảo là máy ngăn xếp
- Máy ngăn xếp là một hệ thống tính toán
 - Sử dụng ngăn xếp để lưu trữ các kết quả trung gian của quá trình tính toán
 - Kiến trúc đơn giản
 - Bộ lệnh đơn giản
- Máy ngăn xếp có hai vùng bộ nhớ chính
 - **Khởi lệnh:**
 - Chứa mã thực thi của chương trình
 - **Ngăn xếp:**
 - Lưu trữ các kết quả trung gian

2. Máy ngăn xếp

PC, B, T là các thanh ghi của máy



2. Máy ngăn xếp → Thanh ghi

- **PC (program counter):**
 - Con trỏ lệnh trỏ tới lệnh hiện tại đang thực thi trên bộ đếm chương trình
- **B (base):**
 - Con trỏ trỏ tới địa chỉ cơ sở của **vùng nhớ cục bộ**. Các biến cục bộ được truy xuất gián tiếp qua con trỏ này
- **T (top);**
 - Con trỏ, trỏ tới đỉnh của ngăn xếp

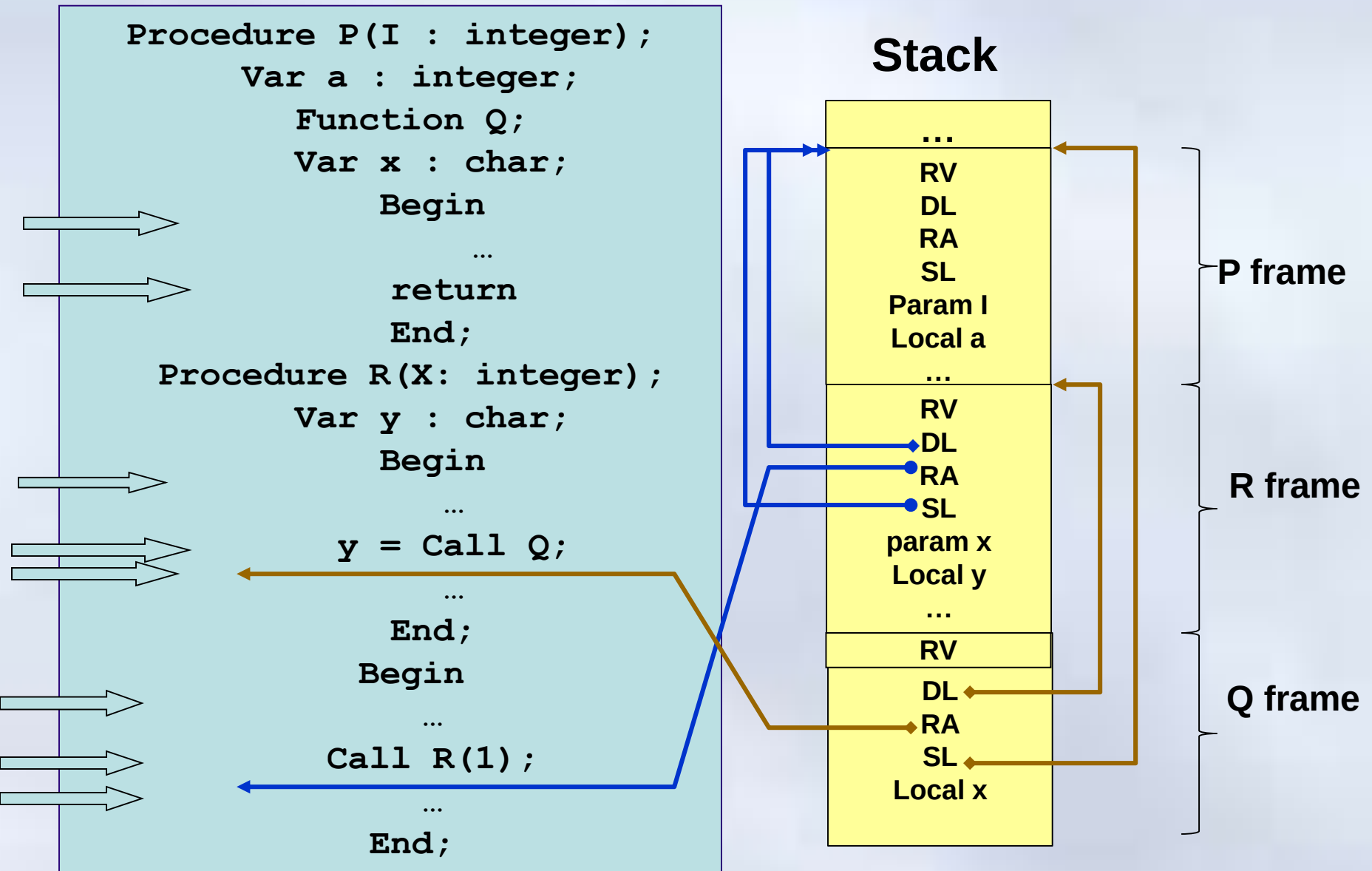
2. Máy ngăn xếp → Bản hoạt động

- Không gian nhớ cấp phát cho mỗi chương trình con (*hàm/thủ tục/chương trình chính*) khi chúng được kích hoạt
 - Lưu giá trị tham số
 - Lưu giá trị biến cục bộ
 - Lưu các thông tin quan trọng khác:
 - RV, DL, RA, SL
- Một chương trình con có thể có nhiều bản hoạt động

2. Máy ngăn xếp → Bản hoạt động (stack frame)

- RV (*return value*):
 - Lưu trữ giá trị trả về cho mỗi hàm
- DL (*dynamic link*):
 - Địa chỉ cơ sở của bản hoạt động của chương trình con gọi tới nó (caller).
 - Được sử dụng để khôi phục ngữ cảnh của chương trình gọi (caller) khi chương trình được gọi (called) kết thúc
- RA (*return address*):
 - Địa chỉ lệnh quay về khi kết thúc chương trình con
 - Sử dụng để tìm tới lệnh tiếp theo của caller khi called kết thúc
- SL (*static link*):
 - Địa chỉ cơ sở của bản hoạt động của chương trình con bao ngoài chương trình được gọi
 - Sử dụng để truy nhập các biến phi cục bộ

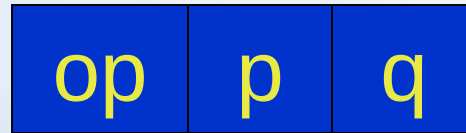
Máy ngăn xếp → Bản hoạt động → Ví dụ



2. Máy ngăn xếp → Lệnh

- Lệnh máy có dạng : **Op p q**
 - Op : Mã lệnh
 - p, q : Các toán hạng.
- Các toán hạng có thể tồn tại đầy đủ, có thể chỉ có 1 toán hạng, có thể không tồn tại
- Ví dụ
 - J** 1 % Nhảy đến địa chỉ 1
 - LA** 0, 4 % Nạp địa chỉ từ số 0+4 lên đỉnh stack
 - HLT** %Kết thúc chương trình

2. Máy ngăn xếp → Bộ lệnh (1/5)



LA	Load Address	$t := t + 1; \quad s[t] := \text{base}(p) + q;$
LV	Load Value	$t := t + 1; \quad s[t] := s[\text{base}(p) + q];$
LC	Load Constant	$t := t + 1; \quad s[t] := q;$
LI	Load Indirect	$s[t] := s[s[t]];$
INT	Increment T	$t := t + q;$
DCT	Decrement T	$t := t - q;$

2. Máy ngăn xếp → Bộ lệnh (2/5)

op	p	q
----	---	---

J	Jump	pc:=q;
FJ	False Jump	if s[t]=0 then pc:=q; t:=t-1;
HLT	Halt	Halt
ST	Store	s[s[t-1]]:=s[t]; t:=t-2;
CALL	Call	s[t+2]:=b; s[t+3]:=pc; s[t+4]:=base(p); b:=t+1; pc:=q;
EP	Exit Procedure	t:=b-1; pc:=s[b+2]; b:=s[b+1];
EF	Exit Function	t:=b; pc:=s[b+2]; b:=s[b+1];

2. Máy ngăn xếp → Bộ lệnh (3/5)

op	p	q
----	---	---

RC	Read Character	read one character into $s[s[t]]$; $t:=t-1$;
RI	Read Integer	read integer to $s[s[t]]$; $t:=t-1$;
WRC	Write Character	write one character from $s[t]$; $t:=t-1$;
WRI	Write Integer	write integer from $s[t]$; $t:=t-1$;
WLN	New Line	CR & LF

2. Máy ngăn xếp → Bộ lệnh (4/5)

op	p	q
----	---	---

ADD	Add	$t := t - 1; s[t] := s[t] + s[t+1];$
SUB	Subtract	$t := t - 1; s[t] := s[t] - s[t+1];$
MUL	Multiply	$t := t - 1; s[t] := s[t] * s[t+1];$
DIV	Divide	$t := t - 1; s[t] := s[t] / s[t+1];$
NEG	Negative	$s[t] := -s[t];$
CV	Copy ^{Top} of Stack	$s[t+1] := s[t]; t := t + 1;$

2. Máy ngăn xếp → Bộ lệnh (5/5)

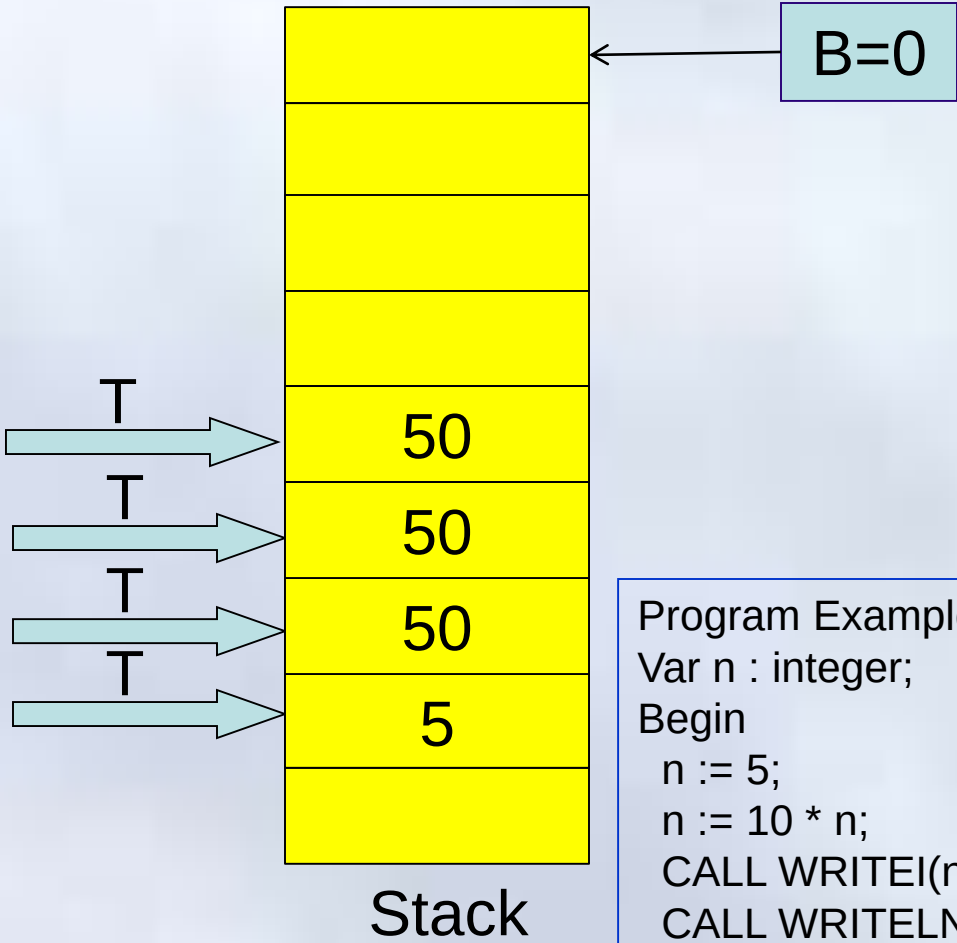
op	p	q
----	---	---

EQ	Equal	$t := t - 1;$ if $s[t] = s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$
NE	Not Equal	$t := t - 1;$ if $s[t] \neq s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$
GT	Greater Than	$t := t - 1;$ if $s[t] > s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$
LT	Less Than	$t := t - 1;$ if $s[t] < s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$
GE	Greater or Equal	$t := t - 1;$ if $s[t] \geq s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$
LE	Less or Equal	$t := t - 1;$ if $s[t] \leq s[t + 1]$ then $s[t] := 1$ else $s[t] := 0;$

2. Máy ngăn xếp → Ví dụ

PC →	0	J 1
PC →	1	INT 5
PC →	2	LA 0, 4
PC →	3	LC 5
PC →	4	ST
PC →	5	LA 0,4
PC →	6	LC 10
PC →	7	LV 0, 4
PC →	8	MUL
PC →	9	ST
PC →	10	LV 0,4
PC →	11	WRI
PC →	12	WLN
PC →	13	HLT

T=-1 →



```
Program Example1;  
Var n : integer;  
Begin  
  n := 5;  
  n := 10 * n;  
  CALL WRITEI(n);  
  CALL Writeln;  
End.
```

2. Máy ngăn xếp → Xây dựng máy ảo

Mã lệnh máy

```
typedef enum {
    OP_LA,    // Load Address:
    OP_LV,    // Load Value:
    OP_LC,    // load Constant
    OP_LI,    // Load Indirect
    OP_INT,   // Increment t
    OP_DCT,   // Decrement t
    OP_J,     // Jump
    OP_FJ,    // False Jump
    OP_HL,    // Halt
    OP_ST,    // Store
    OP_CALL,  // Call
    OP_EP,    // Exit Procedure
    OP_EF,    // Exit Function
```

```
    OP_RC,    // Read Char
    OP_RI,    // Read Integer
    OP_WRC,   // Write Char
    OP_WRI,   // Write Int
    OP_WLN,   // WriteLN
    OP_ADD,   // Add
    OP_SUB,   // Subtract
    OP_MUL,   // Multiple
    OP_DIV,   // Divide
    OP_NEG,   // Negative
    OP_CV,    // Copy Top
    OP_EQ,    // Equal
    OP_NE,    // Not Equal
    OP_GT,    // Greater
    OP_LT,    // Less
    OP_GE,    // Greater or Equal
    OP_LE,    // Less or Equal
    OP_BP     // Break point.
} OpCode ;
```

2. Máy ngăn xếp → Xây dựng máy ảo

Cấu trúc một lệnh

```
typedef struct{  
    OpCode Op;  
    int p;  
    int q;  
} Instruction;
```

Cấu trúc đoạn mã lệnh

```
Instruction Code [MAX_CODE];
```

Cấu trúc đoạn dữ liệu (stack)

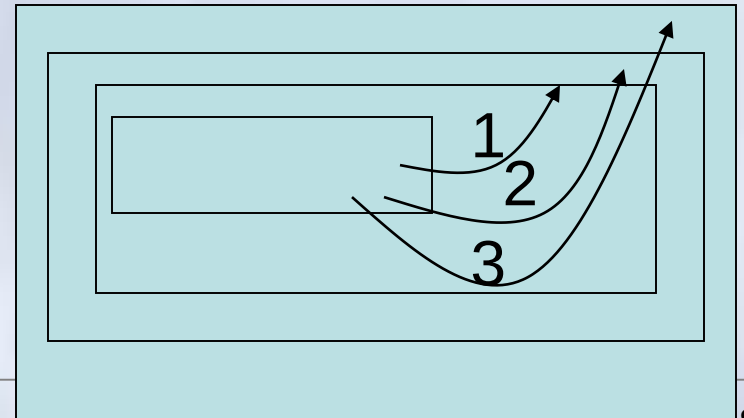
```
int Stack[MAX_DATA];
```

2. Máy ngăn xếp → Xây dựng máy ảo

Bộ thực hiện lệnh

```
void interpreter(){
    int pc = 0, t = -1, b = 0;
    do {
        switch (Code[pc].Op) {
            case OP_LA:  t ++;
                        Stack[t] = base(Code[pc].p) + Code[pc].q;
                        break;
            case OP_LV:  t ++;
                        Stack[t] = Stack[base(Code[pc].p) + Code[pc].q];
                        break;
            case OP_INT:
                        t += Code[pc].q;
                        break;
            case OP_DCT: ...
        }
    } while (true);
}
```

Hàm **base(int L)** dùng để tính địa chỉ cơ sở của chương trình con bao bên ngoài L mức



2. Máy ngăn xếp → Xây dựng máy ảo

```

.....
case OP_CALL:
    Stack[t+2] = b;           // Dynamic Link
    Stack[t+3] = pc;         // Return Address
    Stack[t+4] = base(Code[pc].p); // Static Link
    b = t + 1;               // Base & Result
    pc = Code[pc].q - 1;
    break;
case OP_J:
    pc = Code[pc].q - 1;
    break;
} //switch
pc++;
}while(Exit == 0);
} //intepreter

```

```

int base(int L) {
    int c = b;
    while (L > 0) {
        c = Stack[c + 3];
        L--;
    }
    return c;
}

```

3. Sinh mã đích từ mã trung gian

Mã trung gian là ký pháp Ba lan sau

- Gặp các toán hạng:
 - Sinh mã lệnh đưa giá trị toán hạng lên stack
 - Nếu toán hạng là hằng số: LC
 - Toán hạng là một địa chỉ : LV
- Gặp các toán tử:
 - Sinh mã của toán tử tương ứng

3. Sinh mã đích từ mã trung gian

Ví dụ: biểu thức $1 + 2 * 3 + 4 \Rightarrow 1\ 2\ 3\ * + 4 +$

Sinh mã:

LC 1

LC 2

LC 3

MUL

ADD

LC 4

AD

$(a + b) * (c + d) \Rightarrow a\ b + c\ d + *$
//a, b, c, d là các hằng số

LC a

LC b

ADD

LC c

LC d

ADD

MUL

3. Sinh mã đích từ mã trung gian

Mã trung gian là mã 3 địa chỉ

– **Nguyên tắc:**

- Ánh xạ mỗi câu lệnh mã 3 địa chỉ bội thành dãy các lệnh mã máy tương ứng

– **Giả thiết**

- A là một đối tượng, A.addr là địa chỉ của A trong bảng ký hiệu

– **Chú ý:**

- Các câu lệnh gán 3 địa chỉ có thể chuyển đổi thành dạng hậu tố
- Ví dụ: $a := b + c \Rightarrow a \ b \ c + :=$

3. Sinh mã đích từ mã trung gian

a := b + c

LA a.Addr

LV b.Addr //LC b

LV c.Addr //LC c

ADD

ST

a := b * c

LA a.Addr

LV b.Addr //LC b

LV c.Addr //LC c

MUL

ST

a.Addr : Địa chỉ của a trong đoạn dữ liệu (stack)

- Tính toán dựa vào trường offset trong bảng ký hiệu

3. Sinh mã đích từ mã trung gian

goto L
J f(L)

f(L): Là hàm ánh xạ từ
nhãn L thành một địa chỉ

if a goto L
LV a.Addr
LC 0
EQ
FJ f(L)

if a > 10 goto L
LV a.Addr
LC 10
LE
FJ f(L)

4. Sinh mã đích trực tiếp từ mã nguồn

- Xây dựng lại bảng ký hiệu
 - Bổ sung thông tin cho biến
 - Bổ sung thông tin cho tham số
 - Bổ sung thông tin cho hàm/thủ tục/chương trình
- Sinh mã cho các câu lệnh
 - Câu lệnh gán
 - Câu lệnh rẽ nhánh
 - Câu lệnh lặp
 - Câu lệnh lấy địa chỉ
 - Gọi thủ tục

4. Sinh mã đích trực tiếp từ mã nguồn

Xây dựng lại bảng ký hiệu

- Với biến
 - Vị trí trên bản hoạt động của biến
- Với tham số
 - Vị trí trên bản hoạt động của tham số
 - Vị trí tính từ base của Frame
- Với thủ tục
 - Số lượng tham số

4. Sinh mã cho các lệnh → Biểu thức

Sản xuất	Ba lan sau	Quy tắc sinh mã
$E \rightarrow T\{+T\}$	$E \rightarrow T\{T+\}$	GenT() {genT() ADD}
$T \rightarrow F\{*F\}$	$F \rightarrow F\{F*\}$	GenF() {genF() ML}
$F \rightarrow Id$	$F \rightarrow Id$	LV Id.Addr
$F \rightarrow Num$	$F \rightarrow Num$	LC num
$F \rightarrow (E)$	$F \rightarrow E$	genE()
genE(), genT(), genF(): hàm sinh mã cho ký hiệu E, T, F - genE(), genT(): Là lời gọi tới thủ tục phân tích E, T - genF(): Tùy thuộc vào sản xuất được sử dụng		

$F \rightarrow Id[E] ?$

LA Id.Addr; LC Id.widthType; genE();MUL; ADD; LI;

4. Sinh mã cho các lệnh → Biểu thức

```
void term(){
    factor();
```

```
    while(Token == TIMES || Token == SLASH){
```

```
        Op = Token;
```

```
        Token = getToken();
```

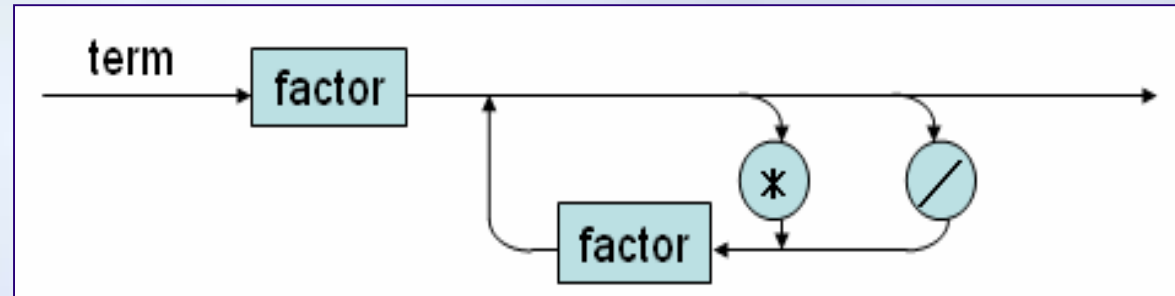
```
        factor();
```

```
        if (Op == TIMES) genCode(OP_ML);
```

```
        else genCode(OP_DV);
```

```
    }
```

```
}
```



4. Sinh mã cho các lệnh → Biểu thức

$$a + 10 * (b + 5)$$

$$\begin{cases} a.Addr = 15 \\ b.Addr = 20 \end{cases}$$

- LV 15
- LC 10
- LV 20
- LC 5
- ADD
- MUL
- ADD

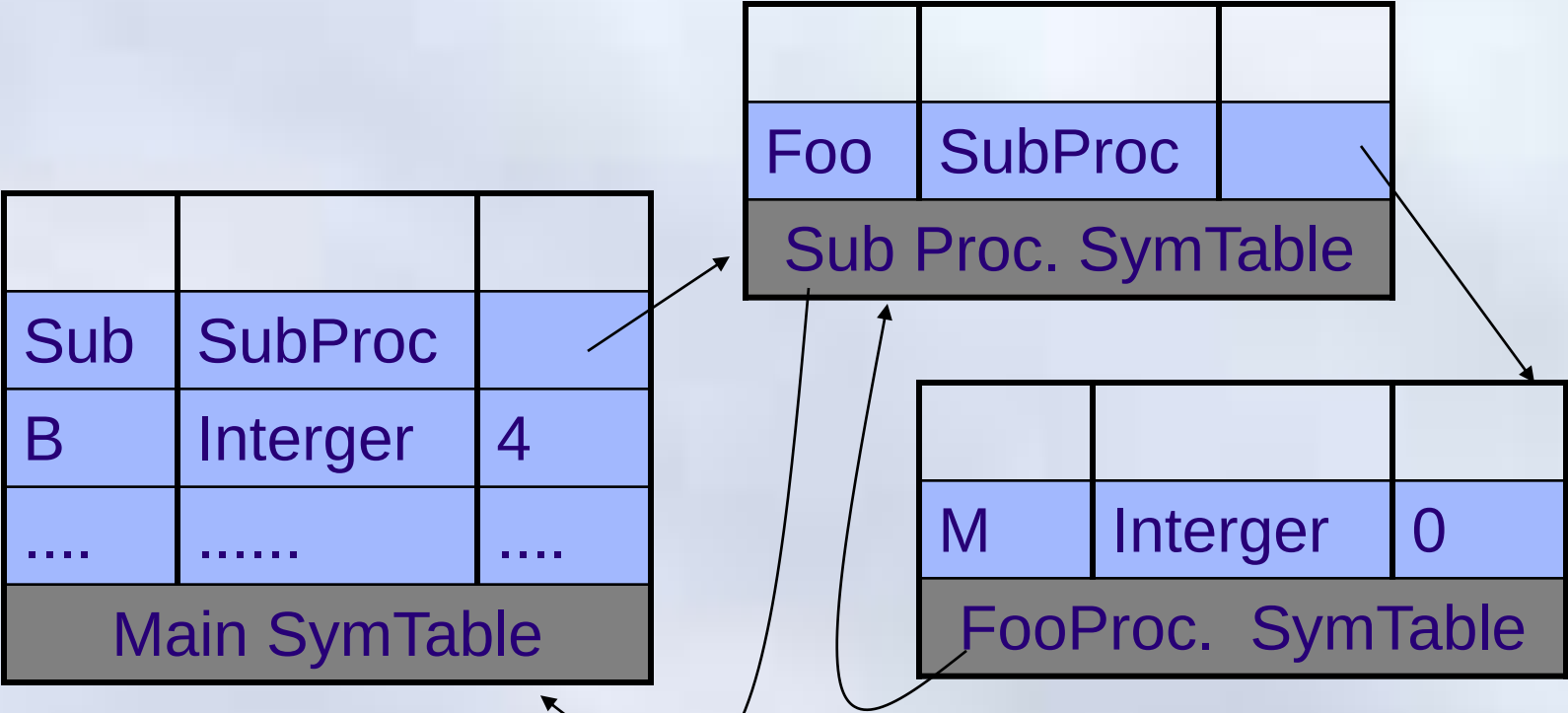
Ba lan sau

a 10 b 5 + * +

4.Sinh mã cho các lệnh → Khái niệm Lvalue

- **Lvalue**

- Danh biểu, có địa chỉ: Biến, tham biến
- Tính toán địa chỉ theo phạm vi và vị trí trong đó



Dùng biến B trong Foo Procedure: Địa chỉ 2, 4

Dùng biến B trong Foo Procedure: Địa chỉ 1, 4

4. Sinh mã cho các lệnh → Lệnh gán

- **Lệnh gán $V := \text{Exp}$**

<code of Lvalue v > // đẩy địa chỉ của v lên stack

<code of exp> // đẩy giá trị của exp lên stack

ST // $S[S[t-1]] = S[t]$

Ví dụ trong thủ tục Foo: **$B := M * 10$**

LA 2, 4

LV 0, 0

LC 10

MUL

ST

4. Sinh mã cho các lệnh → Lệnh rẽ nhánh

If <dk> Then statement

```

<code of dk>    // đẩy giá trị điều kiện dk lên stack
FJ L          //Nhảy có điều kiện, L = 0
<code of statement> //Sinh mã cho statement
L:              //Cập nhật lại nhãn L bằng kích thước đoạn mã
                //được sinh ra bởi statement
  
```

If <dk> Then st1 Else st2

```

<code of dk> // đẩy giá trị điều kiện dk lên stack
FJ L1       //L1 hiện giờ bằng 0
<code of st1>
J L2        //L2 = 0
L1:           //Cập nhật lại nhãn cho L1
<code of st2>
L2:           //Cập nhật lại nhãn cho L2
  
```

4. Sinh mã cho các lệnh → Lệnh lặp while

While <dk> Do statement

L1: <code of dk> // Nhãn L1 xác định

FJ L2 //genFJ(0)

<code of statement>

J L1

L2: //Cập nhật giá trị cho nhãn L2

begin = getCurentCodeAddress()//Xác định L1

compileCondition() //sinh mã cho Condition

Jmp = genFJ(0) //Sinh ra mã lệnh nhảy tới địa chỉ 0

compileStatement()

GenJ(begin)

Jmp.q = getCurrentCodeAddress()//sửa lại địa chỉ nhảy tới

4. Sinh mã cho các lệnh → Lệnh lặp for

For v := exp1 to exp2 do statement

```

<code of Lvalue v>           //Đặt địa chỉ v lên stack
genCV  // Sinh mã CV: nhân đôi địa chỉ của v =Copy địa chỉ V lên stack
<code of exp1>               //Đặt giá trị của biểu thức exp1 lên stack
genST() // Sinh mã lệnh ST: lưu giá trị của exp1 vào giá trị của v
L1:           //Sinh địa chỉ cho nhãn L1: getCurrentCodeAddress()
genCV()
genLI()       // lấy giá trị của v & lưu vào đỉnh stack. Đỉnh stack là giá trị v
<code of exp2> //Sinh mã để đặt giá trị của exp2 lên stack
genLE ()      //Lệnh LE để so sánh 2 giá trị trên đỉnh stack: v và Exp2
genFJ(L2)     //sinh mã lệnh FJ, nhảy tới nhãn L2 chưa xác định
<code of statement> //Sinh mã lệnh cho phần segment
genCV() || genCV(); //Đỉnh stack là địa chỉ của v, Sao chép 2 lần
genLI();           //Đỉnh stack là giá trị của v
genLC(1) || genAD(); //Đặt lên stack 1 rồi cộng với giá trị của v
genST();           // Lưu giá trị mới của v (v+1) vào vị trí của v
genJ( L1)          //nhảy tới nhãn L1 đã tính
L2:               //Tính toán nhãn L2 và cập nhật L2 tương ứng
DCT 1            //Giảm thanh ghi T1 đơn vị do đỉnh stack là v

```

Lấy địa chỉ/giá trị biến

- Khi lấy địa chỉ/giá trị một biến cần tính đến phạm vi của biến
 - Biến cục bộ được lấy từ frame hiện tại
 - Biến phi cục bộ được lấy theo các StaticLink với cấp độ lấy theo “độ sâu” của phạm vi hiện tại so với phạm vi hiện thời

Lấy giá trị của tham số hình thức

- Khi tính toán giá trị của **Factor**
- Cũng cần tính độ sâu như biến
 - Nếu là tham trị: giá trị của tham trị chính là giá trị cần lấy.
 - Nếu là tham biến: giá trị của tham số là địa chỉ của giá trị cần lấy.

Sinh lời gọi hàm/thủ tục

- Lời gọi
 - Thủ tục gộp trong sinh mã lệnh **CallSt**
- Trước khi sinh lời gọi thủ tục cần phải nạp giá trị cho các tham số hình thức bằng cách
 - Tăng giá trị T lên 4 (bỏ qua RV,DL,RA,SL)
 - Sinh mã cho k tham số thực tế
 - Tham trị: `compileExpression()`
 - Tham biến: `compileLValue()`
 - Giảm giá trị T đi $4 + k$
 - Sinh lệnh CALL

Sinh lời gọi hàm/thủ tục→Ví dụ

Procedure Foo(VAR A, B)

CALL Foo(N, A*10)

Lvalue(Ident) trả về địa
chỉ của Ident gồm

- Độ lệch frame
- Offset trong frame

INT 4

LA Lvalue(N)

LV Lvalue(A)

LC 10

MUL

DCT 6

CALL Lvalue(Foo)

Sinh mã cho lệnh CALL (p, q)

Giả sử cần sinh lệnh CALL cho hàm/thủ tục A

Lệnh CALL có hai tham số:

- p: Độ sâu của lệnh CALL, chứa static link.
 $\text{Base}(p) = \text{base}$ của frame chương trình con chứa khai báo của A.
- q: Địa chỉ lệnh mới
 $q + 1 =$ địa chỉ đầu tiên của dãy lệnh cần thực hiện khi gọi A.

CALL (p, q)

```

s[t+2]:=b;           // Lưu lại dynamic link
s[t+3]:=pc;          // Lưu lại return address
s[t+4]:=base(p);     // Lưu lại static link
b:=t+1;              // Base mới và return value
pc:=q;               // địa chỉ lệnh mới
  
```

Hoạt động khi thực hiện lệnh CALL (p, q)

- Thực hiện các lệnh và stack biến đổi tương ứng.
- Khi kết thúc
 - Thủ tục (lệnh **EP**): toàn bộ frame được giải phóng, con trỏ **T** đặt lên đỉnh frame cũ.
 - Hàm (lệnh **EF**): frame được giải phóng, chỉ chứa giá trị trả về tại offset 0, con trỏ **T** đặt lên đầu frame hiện thời (offset 0).

Chương 5: Sinh mã

1. Sinh mã trung gian

2. Sinh mã đích

3. Tối ưu mã

Giới thiệu

Yêu cầu

- Chương trình sau khi tối ưu phải tương đương
- Tốc độ thực hiện trung bình tăng
- Hiệu quả đạt được tương xứng với công sức

Có thể tối ưu mã vào lúc nào

- Mã nguồn
 - Do người lập trình (giải thuật)
- Mã trung gian
- Mã đích

Tối ưu mã cục bộ

- Nguyên tắc
 - Xem xét một dãy lệnh trong mã đích và thay thế chúng bằng những đoạn mã ngắn hơn và hiệu quả hơn
- Xu hướng chính
 - Loại bỏ lệnh dư thừa
 - Thông tin dòng điều khiển
 - Loại bỏ biểu thức con chung
 - Tính toán giá trị hằng
 - Giảm chi phí tính toán
 - Lan truyền biến gán.....

Loại bỏ lệnh dư thừa

- **Mã không đến được**

goto L2

$x := x + 1$ $\leftarrow$ Không cần

L2:....

- **Mã chết**

$x := 32$

$y := x + y$

Nếu x không được dùng trong những lệnh tiếp theo, có thể chuyển thành:

$y := 32 + y$

Thông tin dòng điều khiển

goto L1

...

L1: goto L2 ← Không cần

chuyển thành

goto L2

Loại bỏ biểu thức con chung

- Ví dụ câu lệnh $a[i+1] = b[i+1]$
- Sinh ra mã

$$t1 = i+1$$

$$t2 = b[t1]$$

$$a[t1] = t2$$

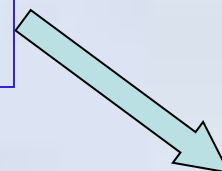
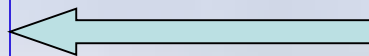
$$a[t3] = t2$$

← Không cần

Tính giá trị hằng

 $x := 32$

trở thành

 $x := 64$ $x := x + 32$ $i = 4$ $t1 = i + 1$ $t2 = b[t1]$ $a[t1] = t2$  $i = 4$ $t1 = 5$ $t2 = b[t1]$ $a[t1] = t2$  $i = 4$ $t1 = 5$ $t2 = b[5]$ $a[5] = t2$  $i = 4$ $t2 = b[5]$ $a[5] = t2$

Giảm chi phí tính toán

- Tối ưu vòng lặp**

Chuyển những đoạn mã bất biến ra ngoài vòng lặp:

```
while (i <= limit - 2)
```

$\Rightarrow$

```
t := limit - 2
```

```
while (i <= t)
```

- Tối ưu toán học**

$x := x + 0$ $\leftarrow$ Không cần

$A := \text{sqrt}(x)$ $\rightarrow A := x * x$

$x := x * 2$ $\rightarrow x := x + x$

$x := x * 8$ $\rightarrow x := x << 3$

Lan truyền biến gán

```
tmp2 = tmp1 ;
```

```
tmp3 = tmp2 * tmp1;
```

```
tmp4 = tmp3 ;
```

```
tmp5 = tmp3 * tmp2 ;
```

```
c = tmp5 + tmp4 ;
```



```
tmp3 = tmp1 * tmp1;
```

```
tmp5 = tmp3 * tmp1 ;
```

```
c = tmp3 + tmp5 ;
```

Khối cơ bản (basic block)

- Khái niệm
 - Chuỗi các lệnh kế tiếp nhau trong đó dòng điều khiển đi vào lệnh đầu tiên của khối và ra ở lệnh cuối cùng của khối mà không bị dừng hoặc rẽ nhánh.
- Ví dụ
$$\begin{aligned}t1 &:= a * a \\t2 &:= a * b \\t3 &:= 2 * t2 \\t4 &:= t1 + t2 \\t5 &:= b * b \\t6 &:= t4 + t5\end{aligned}$$

Giải thuật phân chia các khối cơ bản

- **Input:**
 - Dãy lệnh ba địa chỉ.
- **Output:**
 - DS các khối cơ bản với mã ba địa chỉ của từng khối
- **Phương pháp:**
 - Xác định tập các lệnh đầu, của từng khối cơ bản
 - i. Lệnh đầu tiên của chương trình là lệnh đầu.
 - ii. Bất kỳ lệnh nào là đích nhảy đến của các lệnh GOTO có hoặc không có điều kiện là lệnh đầu
 - iii. Bất kỳ lệnh nào đi sau lệnh GOTO có hoặc không có điều kiện là lệnh đầu
 - Với mỗi lệnh đầu, khối cơ bản bao gồm nó và tất cả các lệnh tiếp theo không phải là lệnh đầu hay lệnh kết thúc chương trình

Giải thuật phân chia các khối cơ bản → Ví dụ

```
1.  prod := 0
2.  i := 1
3.  t1 := 4 * i
4.  t2 := a[t1]
5.  t3 := 4 * i
6.  t4 := b[t3]
7.  t5 := t2 * t4
8.  t6 := prod + t5
9.  prod := t6
10. t7 := i + 1
11. i := t7
12. if i <= 20 goto 3
```

- Lệnh (1) là lệnh đầu theo quy tắc i,
- Lệnh (3) là lệnh đầu theo quy tắc ii
- Lệnh sau lệnh (12) là lệnh đầu theo quy tắc iii.
- Các lệnh (1) và (2) tạo nên khối cơ bản thứ nhất.
- Lệnh (3) đến (12) tạo nên khối cơ bản thứ hai.

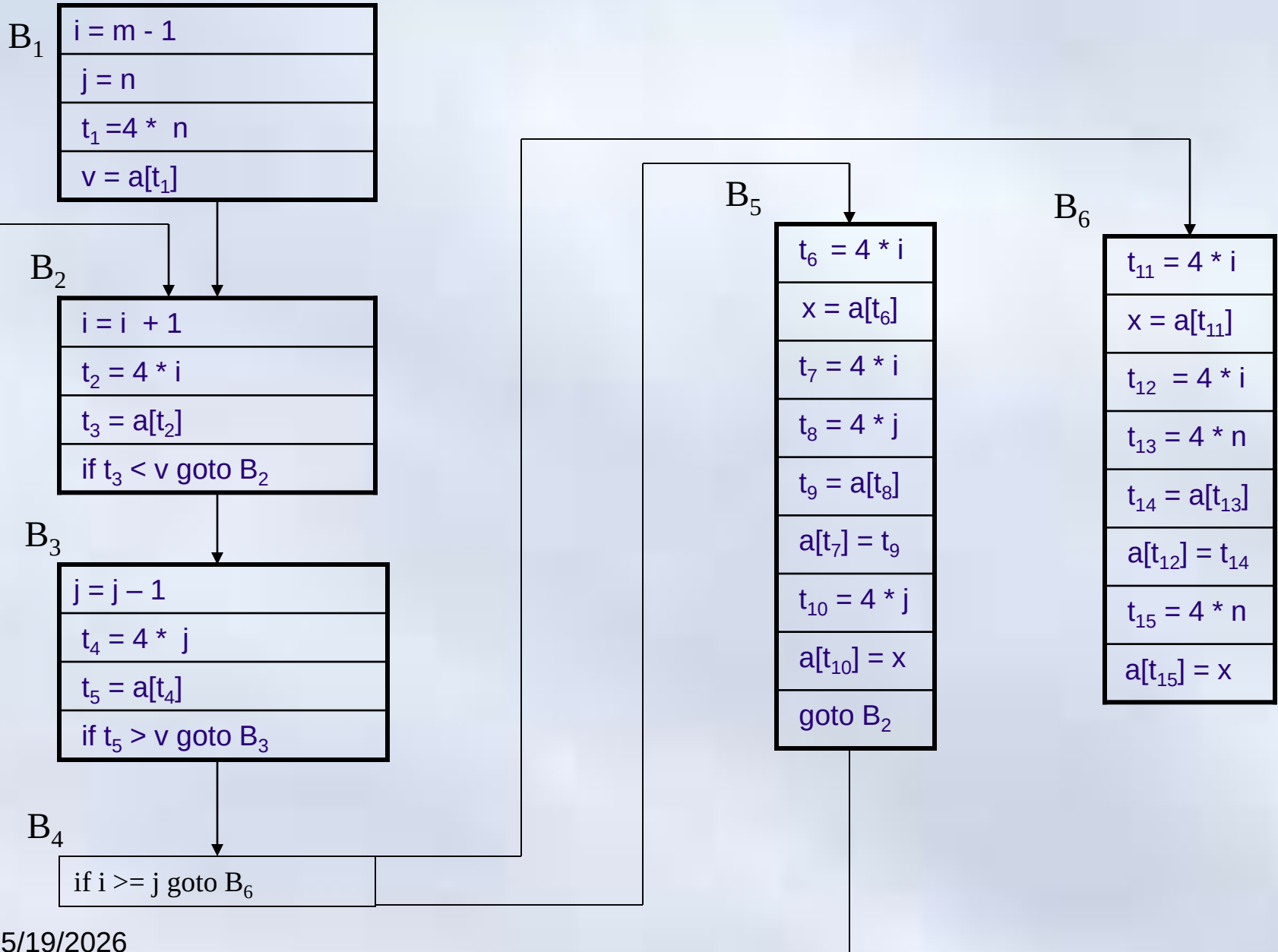
Mã ba địa chỉ của Quick Sort

1	$i = m - 1$
2	$j = n$
3	$t_1 = 4 * n$
4	$v = a[t_1]$
5	$i = i + 1$
6	$t_2 = 4 * i$
7	$t_3 = a[t_2]$
8	if $t_3 < v$ goto (5)
9	$j = j - 1$
10	$t_4 = 4 * j$
11	$t_5 = a[t_4]$
12	if $t_5 > v$ goto (9)
13	if $i \geq j$ goto (23)
14	$t_6 = 4 * i$
15	$x = a[t_6]$

Xác
định
khối cơ
bản

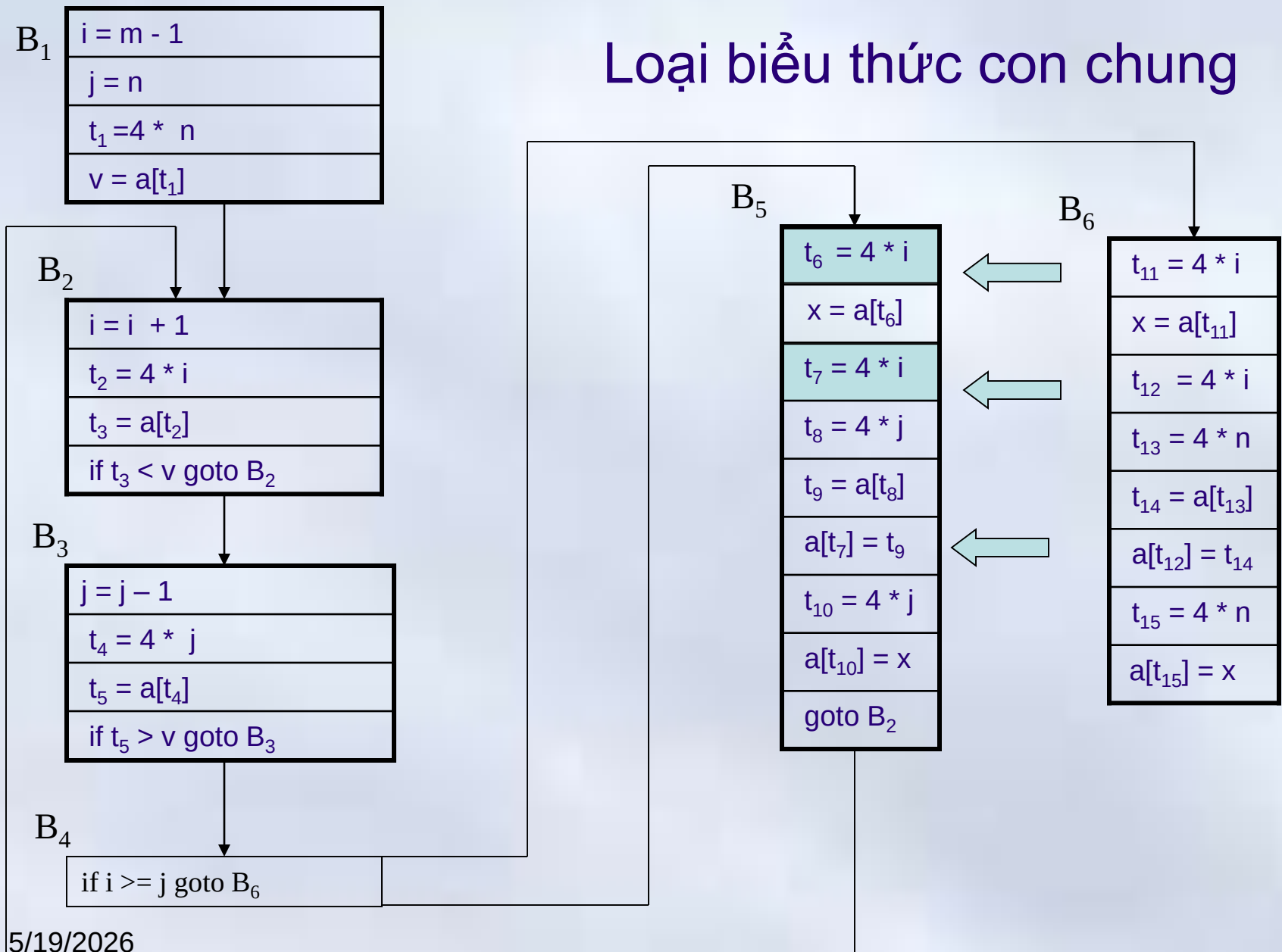
16	$t_7 = 4 * i$
17	$t_8 = 4 * j$
18	$t_9 = a[t_8]$
19	$a[t_7] = t_9$
20	$t_{10} = 4 * j$
21	$a[t_{10}] = x$
22	goto (5)
23	$t_{11} = 4 * i$
24	$x = a[t_{11}]$
25	$t_{12} = 4 * i$
26	$t_{13} = 4 * n$
27	$t_{14} = a[t_{13}]$
28	$a[t_{12}] = t_{14}$
29	$t_{15} = 4 * n$
30	$a[t_{15}] = x$

Đồ thị quan hệ các khối

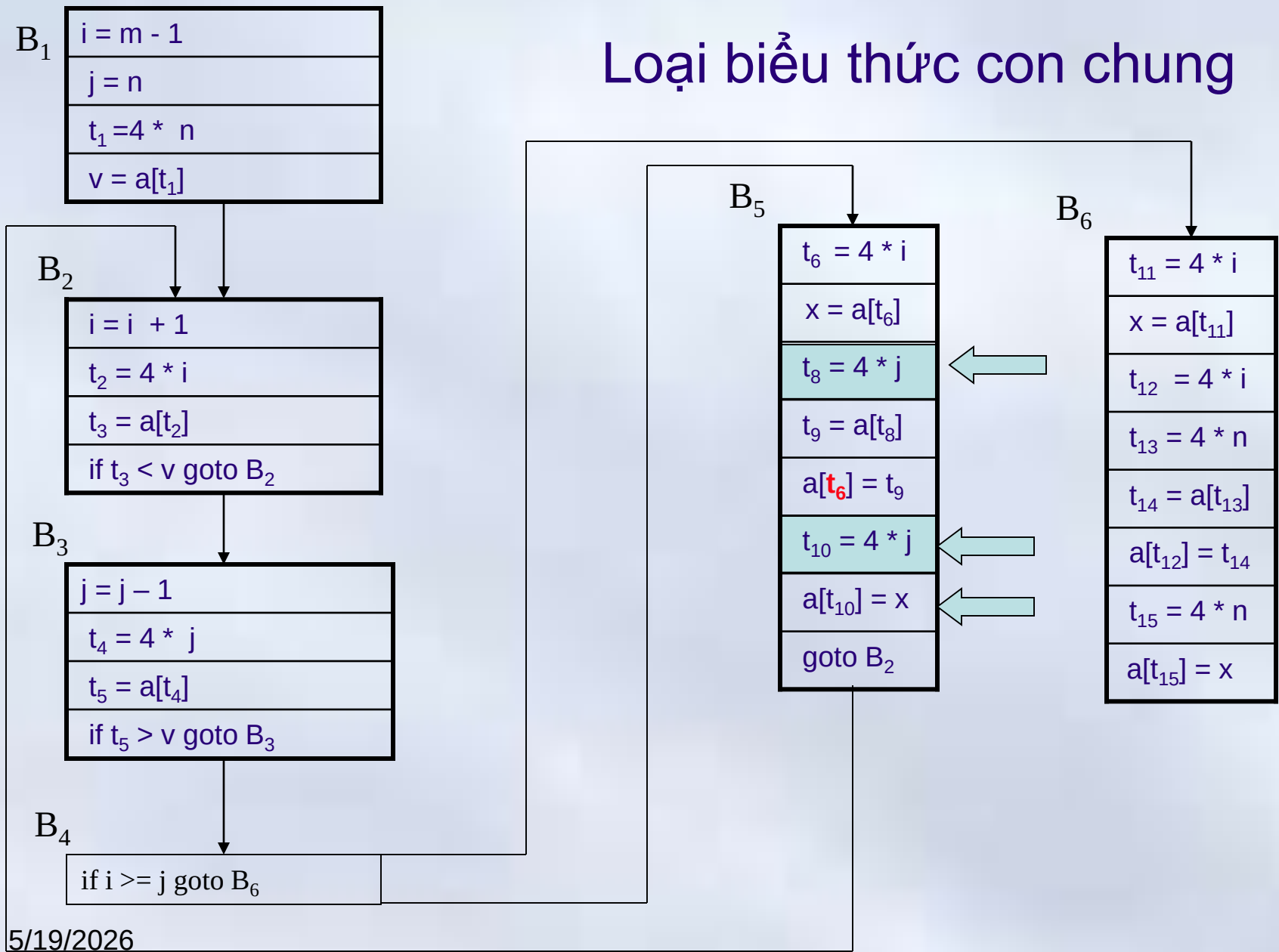


Tối ưu mã → Ví dụ

Loại biểu thức con chung

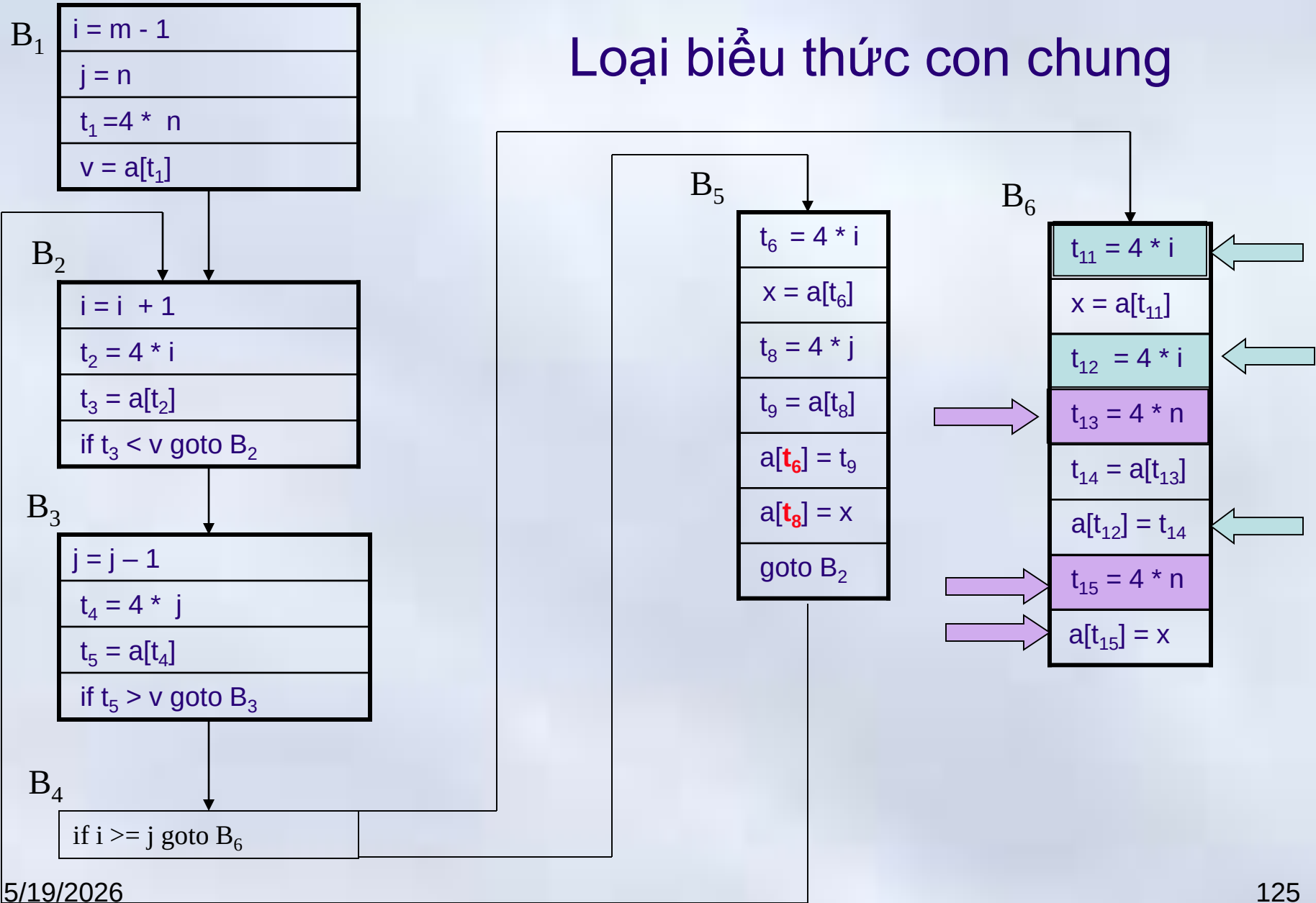


Tối ưu mã → Ví dụ

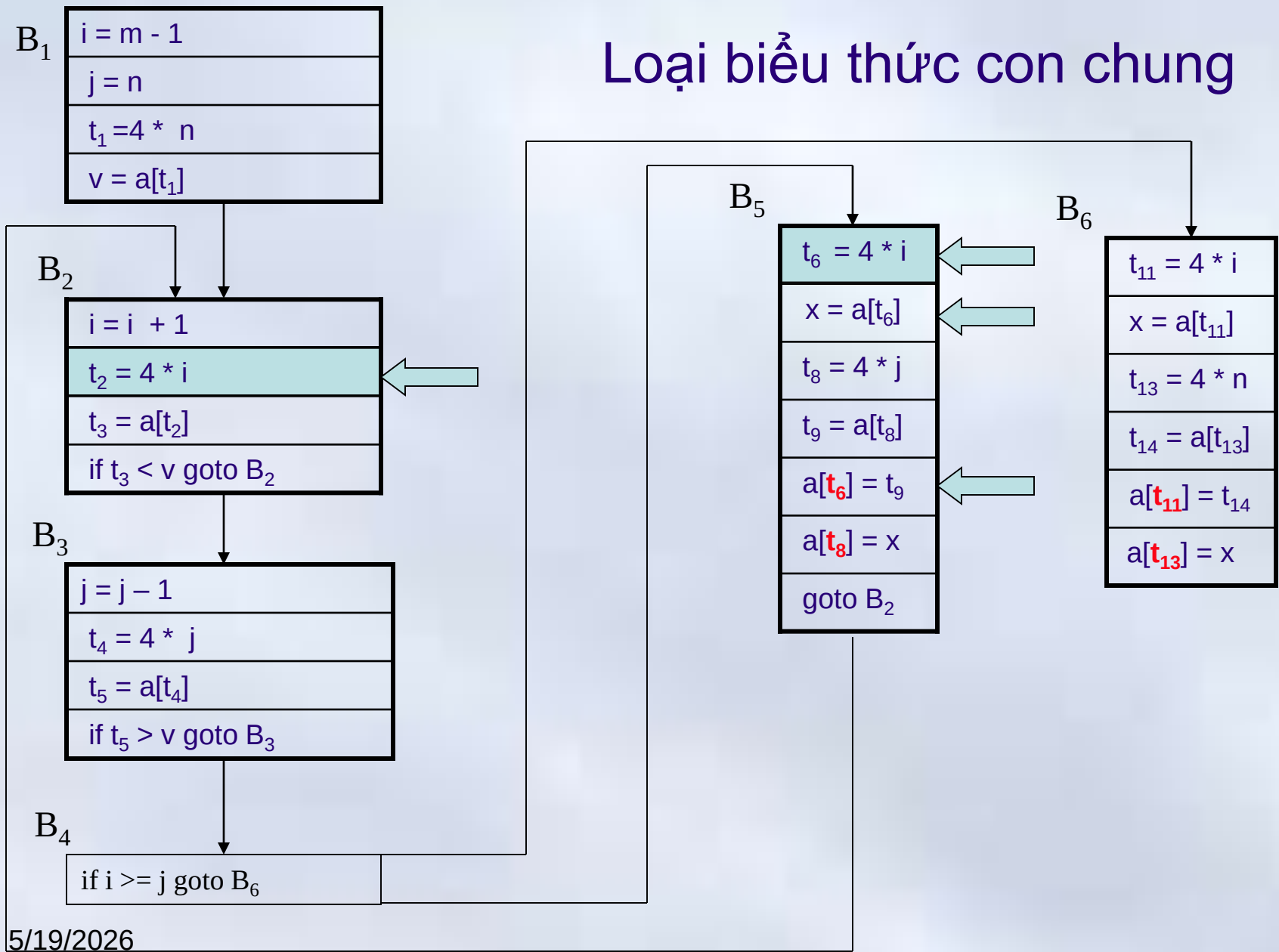


Tối ưu mã → Ví dụ

Loại biểu thức con chung

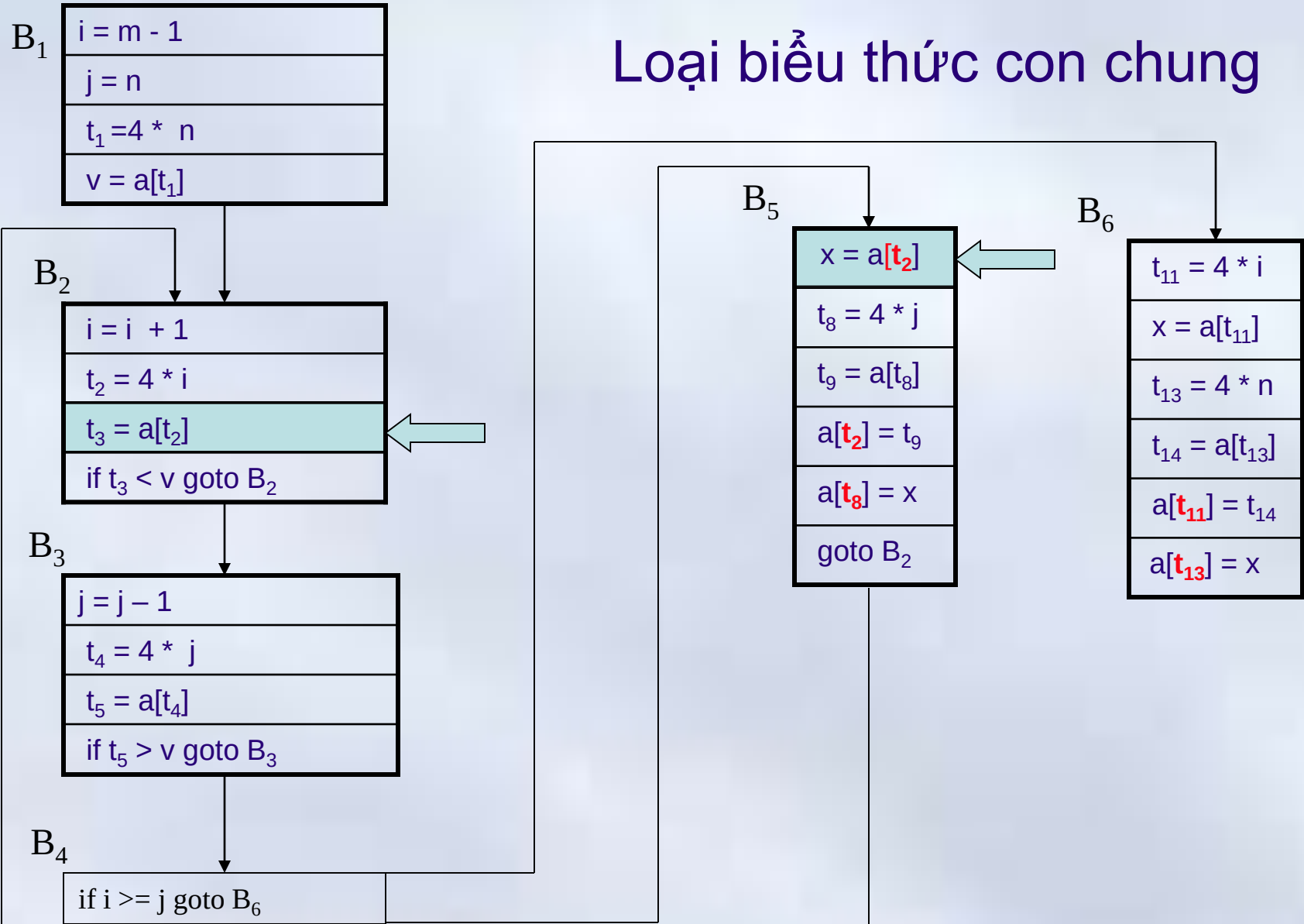


Tối ưu mã → Ví dụ



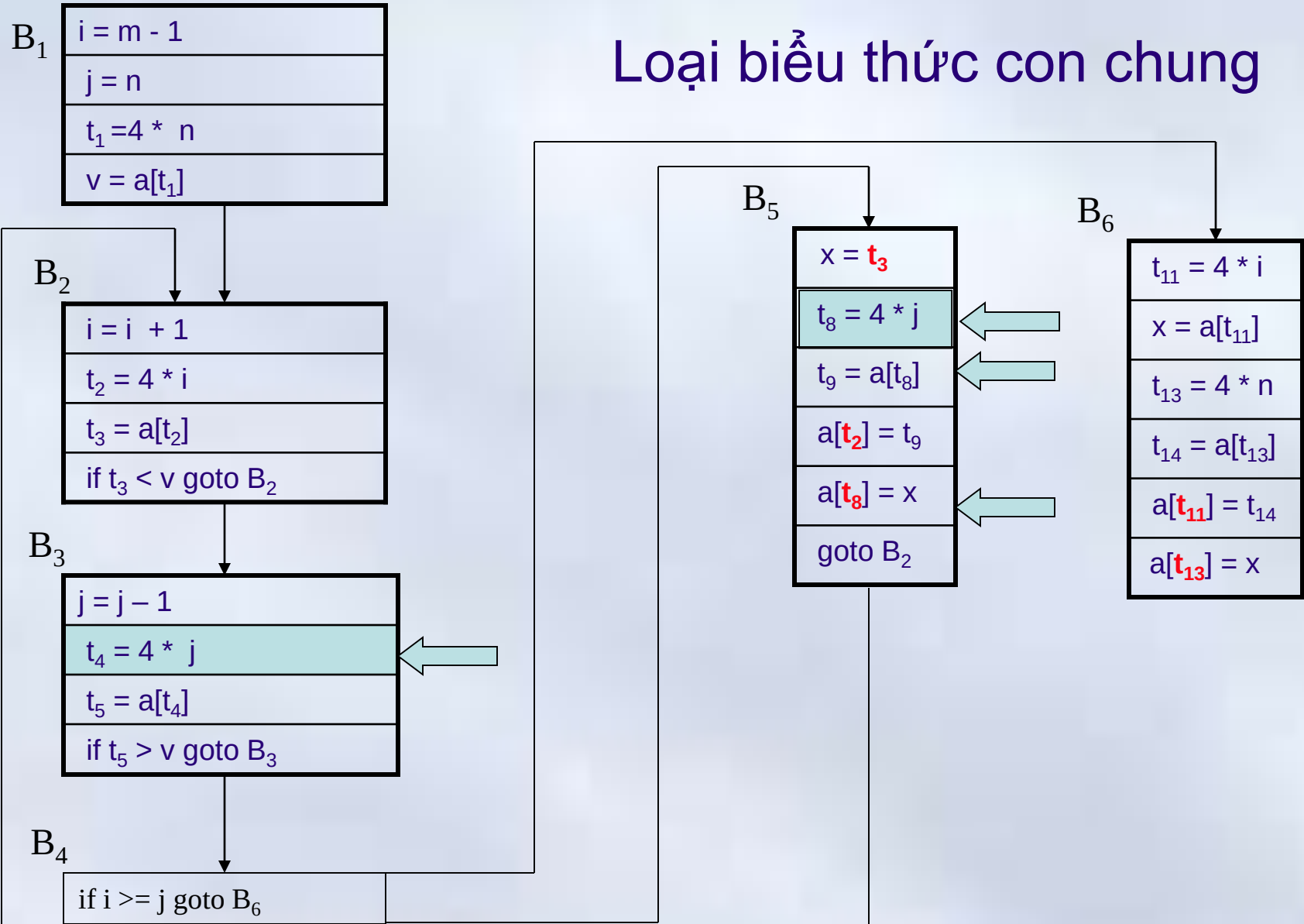
Tối ưu mã → Ví dụ

Loại biểu thức con chung



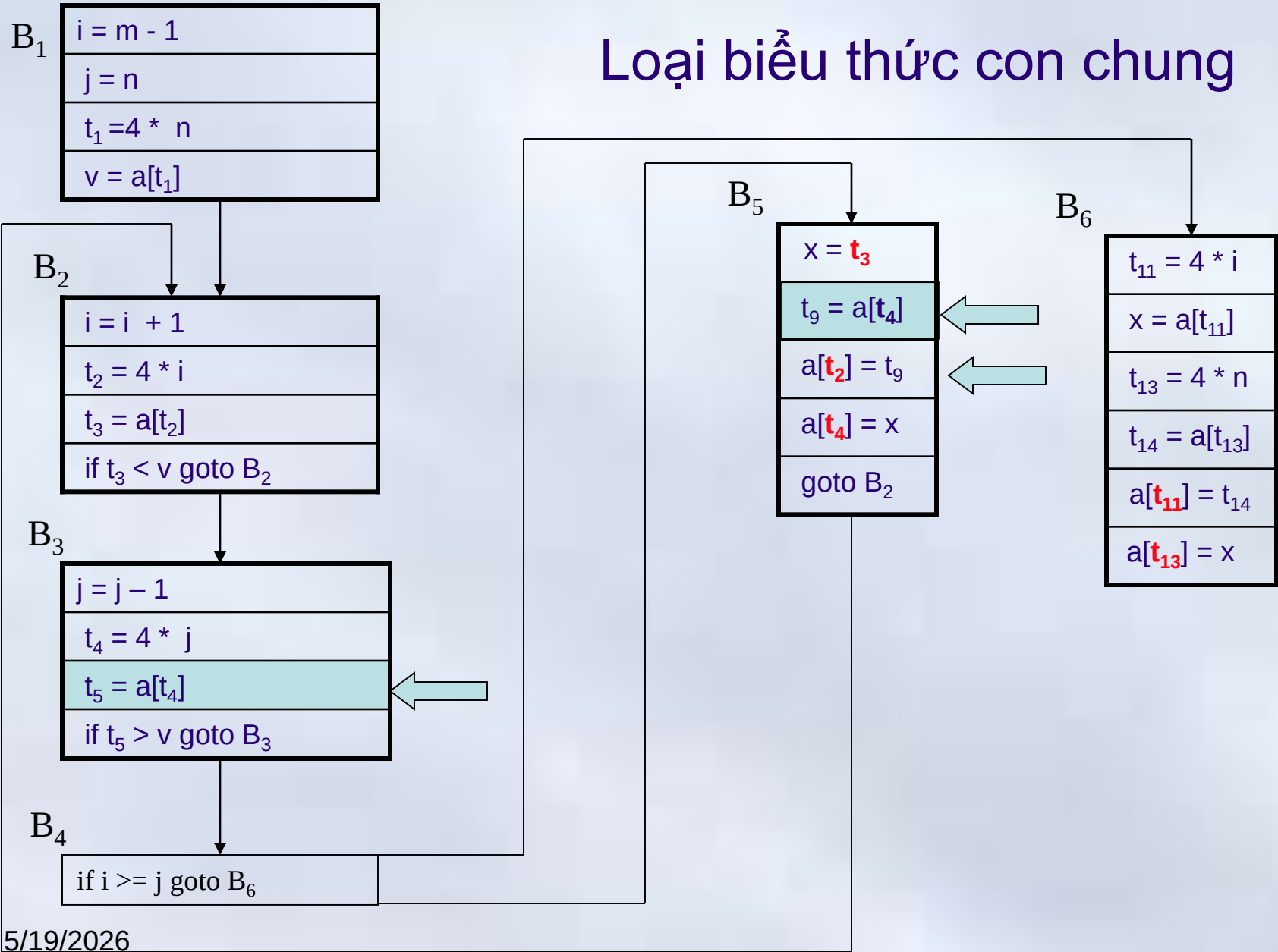
Tối ưu mã → Ví dụ

Loại biểu thức con chung

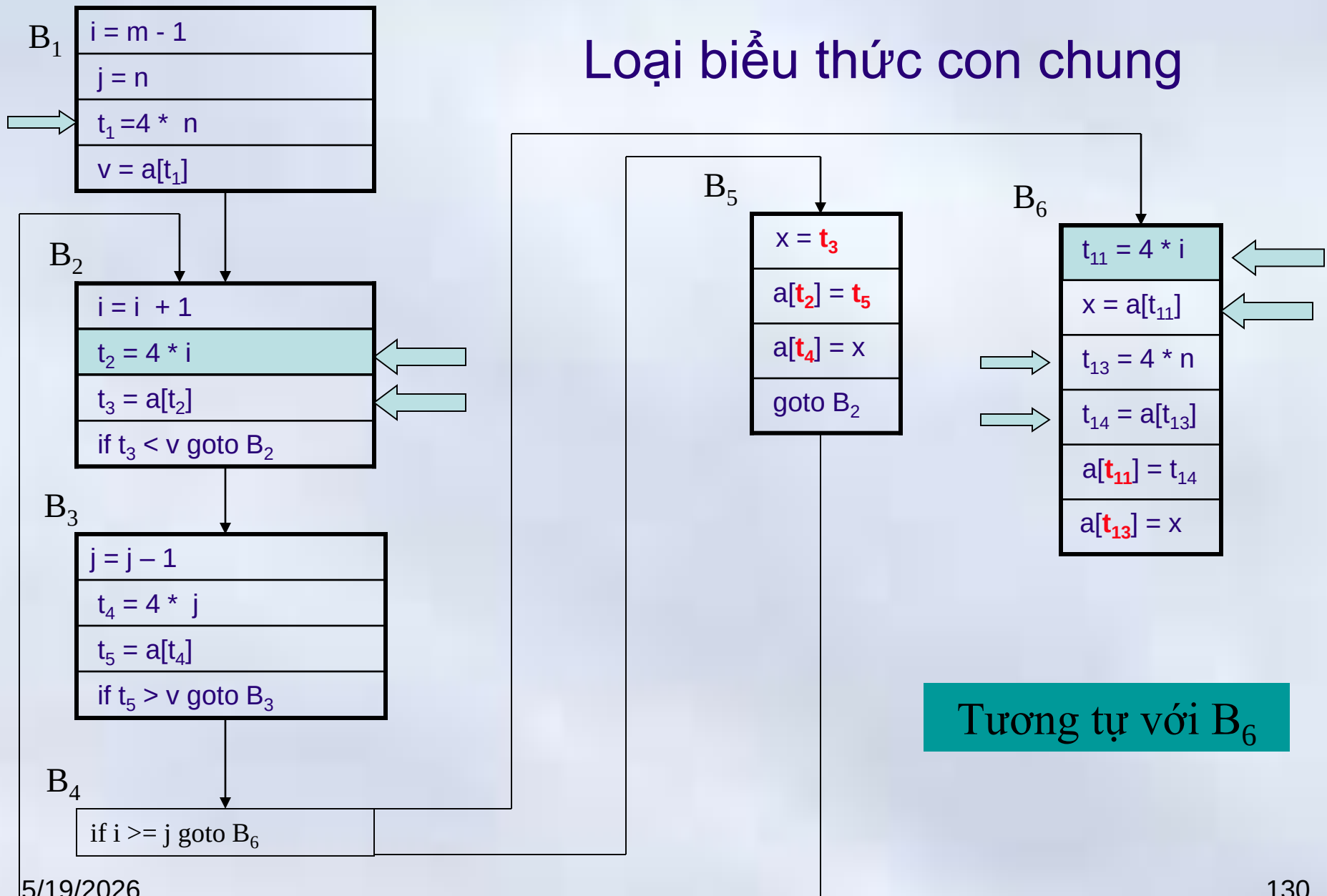


Tối ưu mã → Ví dụ

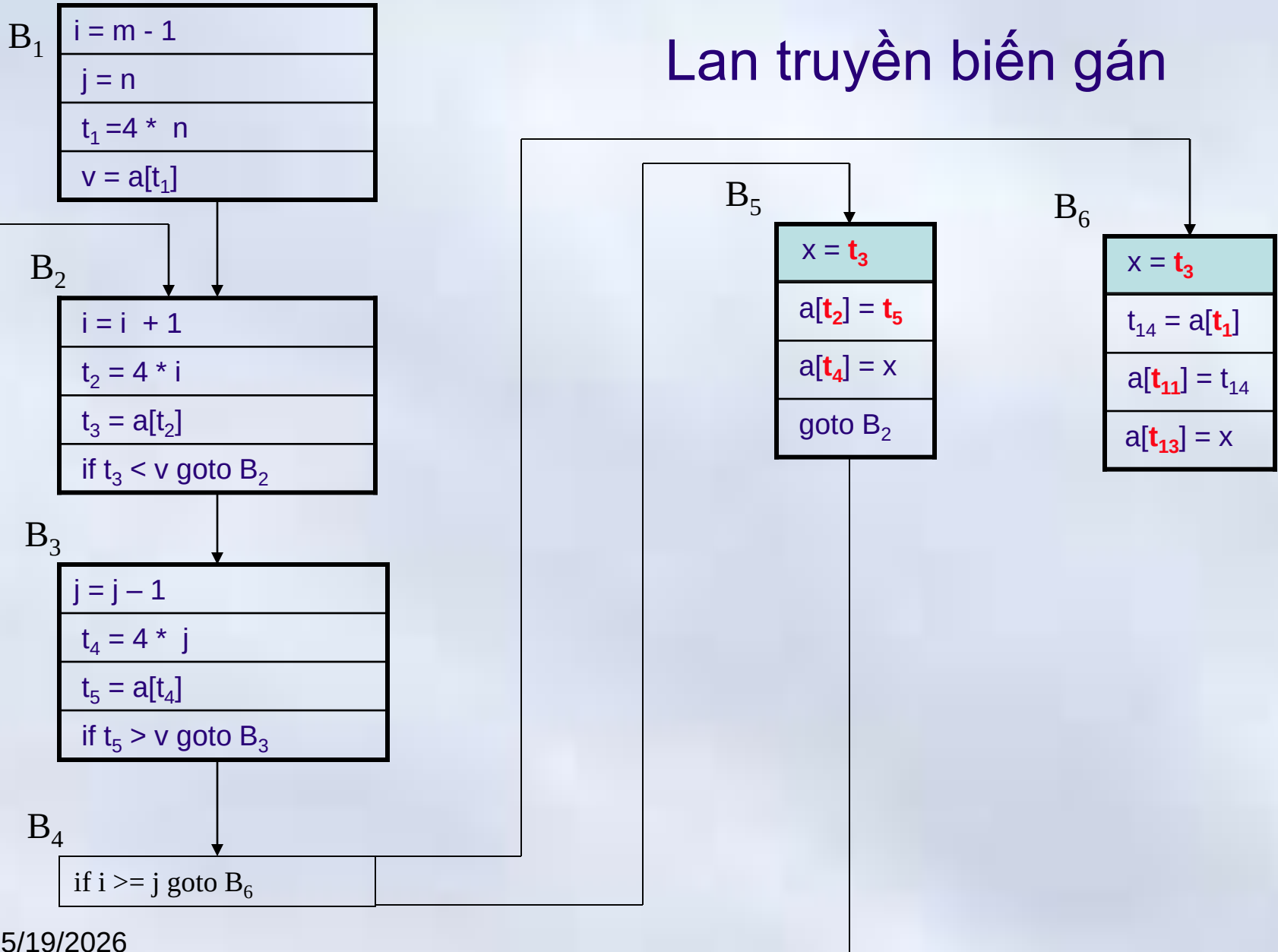
Loại biểu thức con chung



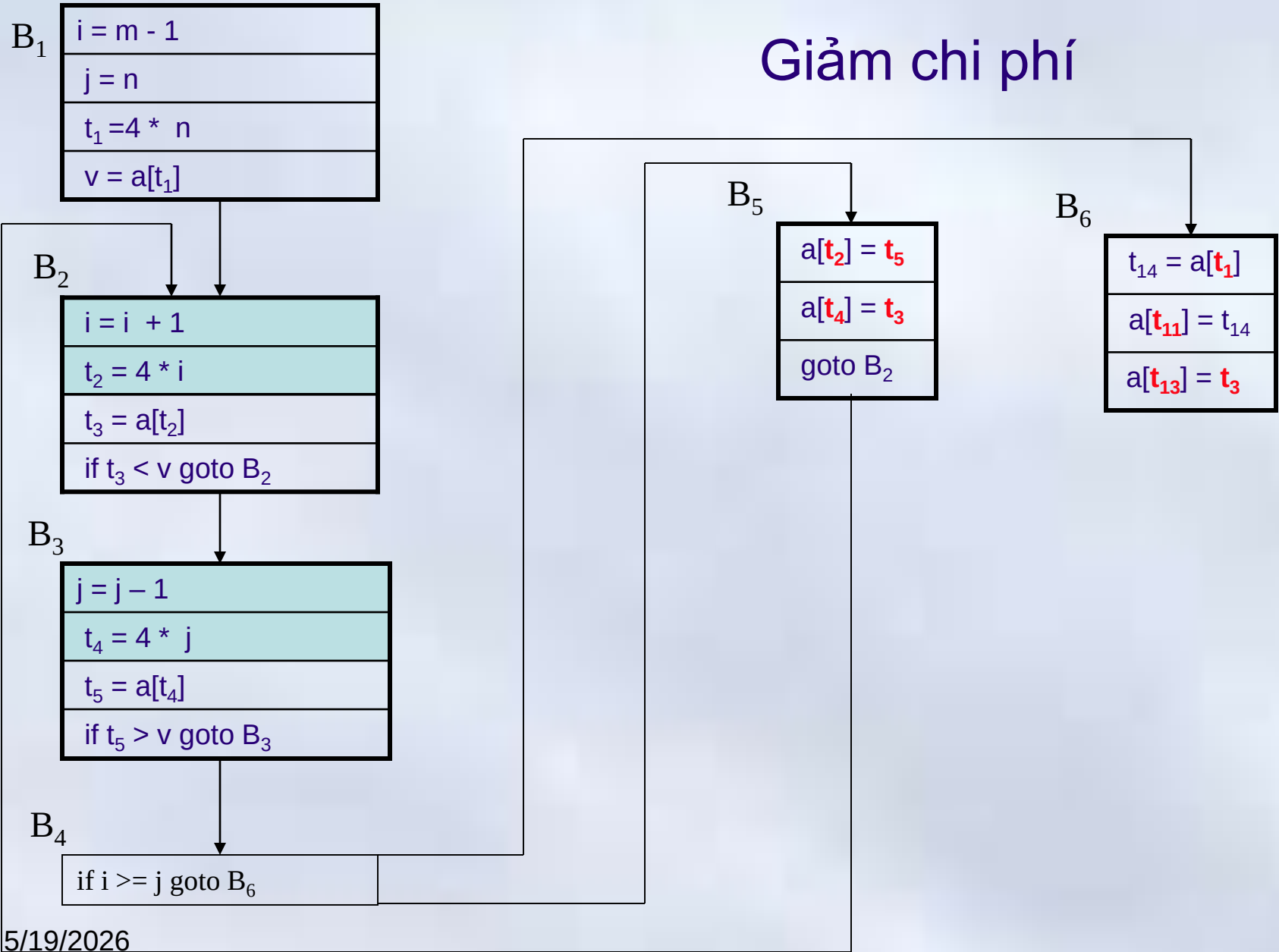
Tối ưu mã → Ví dụ



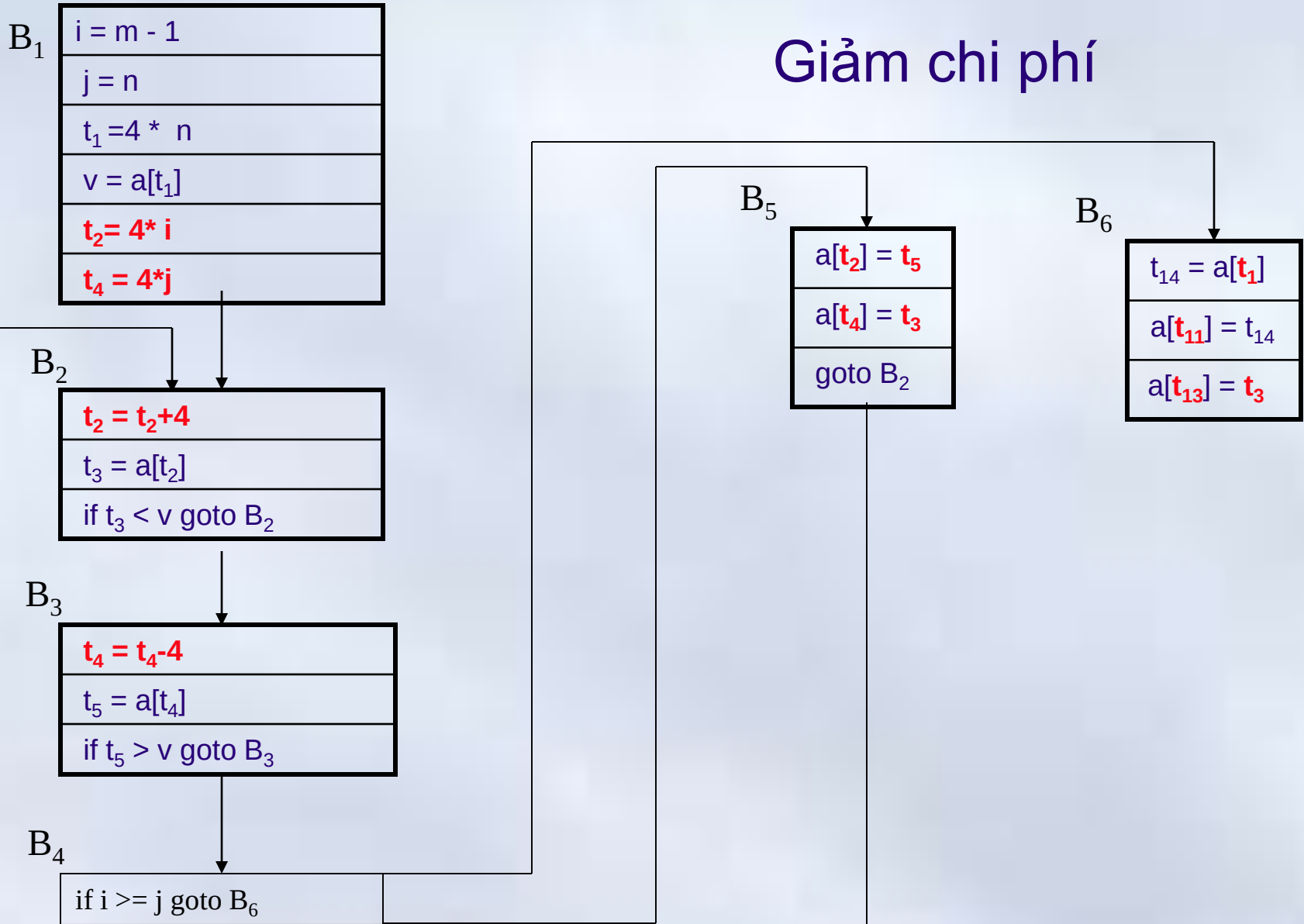
Tối ưu mã → Ví dụ



Tối ưu mã → Ví dụ



Tối ưu mã → Ví dụ



Tối ưu mã đích

- Không tồn tại một giải thuật tổng quát để tìm ra chương trình tối ưu nhất
 - Dùng một số thuật toán đơn giản để tối ưu mã đích
 - Xét ví dụ máy có 1 thanh ghi **R** và tồn tại các lệnh
 - **Load m** → Nạp vào thanh ghi R giá trị m ($R=m$)
 - **Add m** → Cộng vào thanh ghi R giá trị m ($R+=m$)
 - **Mul m** → Nhân vào thanh ghi R giá trị m ($R*=m$)
 - **Store m** → Ghi giá trị thanh ghi R vào địa chỉ m
- Trong đó, **m** có thể là hằng số, biến

Tối ưu mã đích → Phép biến đổi đơn giản

- Phép giao hoán
 - Hai lệnh liên tiếp **Load** α và **Add** β có thể được thay thế bằng **Load** β và **Add** α
 - Hai lệnh liên tiếp **Load** α và **Mul** β có thể được thay thế bằng **Load** β và **Mul** α
- Dãy lệnh **store** α **Load** α sẽ bị hủy bỏ nếu
 - α không được sử dụng hoặc được lưu giá trị mới trước khi dùng (store α)
- Dãy lệnh **Load** α **store** β sẽ bị hủy bỏ nếu
 - Sau đó là lệnh load khác và không có sự thay đổi β từ đó về sau, đồng thời sự sử dụng α được thay bằng β

Tối ưu mã đích → Phép biến đổi đơn giản → Ví dụ

- | | | | |
|------------|------------|--|------------|
| • Load c | • Load c | | |
| • Store #3 | • Store #3 | | |
| • Load b | • Load b | | |
| • Store #1 | • Store #1 | | |
| • Load a | • Load # 1 | | • Load c |
| • Add #1 | • Add a | | • Store #3 |
| • Store #2 | • Store #2 | | • Load b |
| • Load 10 | • Load #2 | | • Add a |
| • Mul #2 | • Mul 10 | | • Mul 10 |
| • Add # 3 | • Add # 3 | | • Add c |
| • Store V | • Store V | | • Store V |